

绿电直连型电氢氨园区优化运行

摘要

随着“双碳”战略深入推进，新能源大规模并网消纳难的问题日益凸显。绿电直连型电氢氨一体化园区通过“绿电—绿氢—绿氨”路径，将新能源发电与化工生产深度耦合，成为解决新能源消纳和化工行业深度脱碳的重要途径。本文针对绿电直连型电氢氨园区的优化运行问题，以某 40MW 风电、64MW 光伏配建电氢氨装置的典型园区为研究对象，建立了从功率平衡分析到储能配置优化的完整数学模型体系，产出了 19 个可量化分析图表及 9 个数据结果文件。

针对问题一，基于功率平衡原理，建立了典型日逐时功率平衡模型，在满负荷连续运行条件下解析计算了购电、售电、绿电指标和吨氨成本。结果表明：36 吨/日产能下绿电比例 (69.21%) 达标，但自发自用比例 (28.16%) 和上网比例 (35.92%) 不达标，主要原因是制氢氨负荷 (20.75MW) 远小于新能源装机容量 (104MW)，大量绿电上网导致自用比例不足，吨氨成本为 4,322 元/吨。

针对问题二，针对电氢氨装置仅能全额开机或停机的离散运行特点，建立了基于边际成本排序的混合整数规划模型，以日净电费最小为目标函数，对各产量水平 (72-36 吨/日) 分别优化了最优生产时段安排。结果表明：72、63、54 吨/日可满足全部绿电指标，45 吨/日及以下因自用比例骤降而不达标。24 种风光场景下，72 吨/日时 18/24 场景全满足，36 吨/日时 0/24 场景全满足，离散调节在低产量下难以满足绿电要求。

针对问题三，针对电氢氨装置功率可连续调节 (10%-100%) 的特性，建立了含 10% 下限约束的线性规划模型，采用贪心分配算法求解各时段最优功率分配。结果表明：连续调节在各产量下均将自用比例稳定在 82% 以上、上网比例控制在 9% 以内，全达标场景数从离散模式的 0-18 个提升至 18-22 个。连续调节以略高的吨氨成本 (36 吨/日时 3,406 元/吨) 换取了绿电指标的显著改善，证明其在绿电直连园区中的推广价值。

针对问题四，针对园区离网运行场景，建立了含储能的离网功率平衡模型和储能 SOC 动态方程。外层枚举储能容量 (0-200MWh)，内层优化逐时调度，通过 Pareto 分析寻求最优配置。结果表明：24 种风光场景下离网平均产能利用率 37.7%，平均可再生利用率 93.7%。在当前储能成本 (1,000 元/kWh) 下最优容量为 0MWh，配储不具经济性。在同等产量下，联网模式吨氨成本 (3,697 元/吨) 低于离网模式 (4,158 元/吨)，联网具有全面经济优势。

针对问题五，从调峰压力、备用容量、电能质量三个角度分析了高渗透率绿电园区的负面影响，从促进消纳、降低碳排放、推动产业链三个角度分析了正面影响。结合前四问数值结果，提出了完善市场机制 (优先级 92%)、差异化电价 (88%)、强制配储 (82%)、技术研发 (78%)、智能调度 (75%) 等五条政策建议，为绿电直连园区发展提供了定量论证依据。

关键字： 绿电直连 电氢氨园区 混合整数规划 储能配置 运行优化

一、问题重述

1.1 问题背景

随着“双碳”战略深入推进，新能源装机规模持续快速增长，但新能源大规模并网消纳难的问题日益凸显，成为制约能源绿色转型的关键瓶颈 [1, 10]。绿电直连项目通过将新能源发电端与用电负荷直接连接，为促进新能源就近消纳提供了新路径。其中，“绿电—绿氢—绿氨”一体化电氢氨化工园区，利用可再生能源电解水制氢再将氢转化为氨，既解决了绿电就地消纳问题，又为化工行业提供了深度脱碳的可行路径 [2]。因此，研究绿电直连型电氢氨园区的优化运行问题，对推动新能源消纳和化工行业绿色转型具有重要的理论价值和现实意义。

本文以某典型绿电直连制氨项目为研究对象，园区包含风电 (40MW)、光伏 (64MW)、碱性电解槽、质子交换膜电解槽、合成氨装置、常规负荷及联网线路。给定 24 种风、光出力组合场景及分时电价等参数，研究园区的优化运行问题。

1.2 问题重述

问题一：典型风光场景下的运行指标分析。给定典型日风光出力及常规负荷曲线，电解槽与合成氨装置满负荷连续运行 (36 吨/日)。要求根据功率平衡计算购售电功率并绘制曲线，计算绿电直连三项指标及吨氨成本，分析达标情况及原因。

问题二：基于离散制氨调节的运行优化。制氨产能增至 72 吨/日，产量从 72 吨/日按 9 吨递减至 36 吨/日，电氢氨装置仅全额开机或停机。要求在典型场景下给出各产量成本最低的生产时段安排及绿电指标，找出最优日产量；在 24 种风光场景下统计分析各指标的分布特征，按全满足、部分满足、全不满足三类统计全年绿电指标状况，绘制成本分布曲线并计算全年总吨氨成本。

问题三：基于连续制氨调节的运行分析。产能 72 吨/日，产量从 72 吨/日递减至 36 吨/日，电氢氨装置功率连续可调 (下限 10%)。要求在 24 种风光场景下计算最优功率调度方案及相应指标和成本，按三类情况统计全年绿电指标，分析各场景运行原因，并与问题二结果对比分析变化情况。

问题四：离网运行分析及储能配置研究。园区离网运行，制氨产能 72 吨/日，功率连续可调 (下限 10%)。要求在尽限利用风光条件下计算各场景产量、成本及产能利用率，分析能源自给状况；给出最大弃电场景的储能最优配置方案；基于相同产量需求对比离网和联网两种运行模式的经济性。

问题五：政策影响分析与建议。分析绿电园区容量渗透率显著提高后对电力系统运行的影响（至少三种利弊），提出推动绿电直连园区发展的政策建议并给出论证依据。

二、问题分析

2.1 问题一的分析

问题一要求计算典型风光场景下园区运行的功率平衡和绿电指标。这是一个确定性的功率平衡核算问题——给定风光出力曲线和常规负荷曲线，在电解槽和合成氨装置满负荷连续运行的条件下，各时段的发电和用电功率均可精确确定。分析思路为：首先逐时计算风电、光伏出力与常规负荷、制氢氨负荷的差值，据此判定各时段需要网购电力还是有余电上网；然后汇总一日内的发电量、用电量、购电量、售电量，计算三个绿电直连指标和吨氨成本；最后将各指标与要求值对比，对不达标指标分析原因。本问题采用解析计算方法，无需建立优化模型。

2.2 问题二的分析

问题二要求园区制氨产能扩至 72 吨/日后，在电氢氨装置仅能全额开机或停机的条件下，寻找各产量水平 (72-36 吨/日) 成本最低的生产时段安排。这是一个离散组合优化问题——每小时的生产决策为 0-1 变量（开机或停机），需要在满足总产量目标的前提下最小化日净电费（购电成本减售电收益）。分析思路为：首先注意到各时段的生产决策相互独立（无启停成本约束），因此可以分别计算每个时段开机的边际成本（相比不开机的费用变化）；然后对所有时段按边际成本从小到大排序，选择成本最低的 k 个时段开机，即可获得严格最优解。对于 24 种风光场景的统计分析，将每种场景作为独立算例重复求解，再按全满足、部分满足、全不满足三类对绿电指标达标情况进行分类统计。

2.3 问题三的分析

问题三在问题二的基础上，将电氢氨装置的运行方式从离散开关改为功率连续可调 (下限 10%)。这是一个连续优化问题——各时段的制氢氨功率为连续变量，在满足总产量约束的前提下，通过优化功率在各时段的分配来最小化日总成本。分析思路为：该问题的目标函数是分段线性凸函数，可采用贪心分配策略求解——首先将功率优先分配给可再生盈余最大的时段（利用免费绿电，仅损失售电机会成本）；若仍有剩余产量需求，再按边际购电成本从小到大的顺序将功率分配到各时段。由于功率可连续调节，可以在各时段精确匹配可再生出力，预期将显著改善绿电指标。最后将计算结果与问题二进行对比，分析连续调节相比离散调节的优势。

2.4 问题四的分析

问题四要求分析园区离网运行时的能源自给状况并研究储能的最优配置。离网运行下园区与电网无电气连接，制氢氨功率必须跟随可再生出力变化，无法从电网购电补充。分析分三个层次：首先，在无储能条件下，计算各风光场景下制氢氨功率对可再生出力的跟随情况，统计产量、产能利用率和弃电率；其次，针对最大弃电场景，采用双层优化方法——外层枚举候选储能容量，内层在给定容量下优化逐时充放电调度，通过绘制 Pareto 曲线（吨氨成本与弃电率的权衡）寻找最优储能配置；最后，在相同产量需求下，对比离网模式与联网模式（调用问题三连续调节模型）的经济性，包括吨氨成本、年总产量等指标。

2.5 问题五的分析

问题五是一个定性与定量相结合的政策分析问题。基于前四问的数值结果和电力系统运行的基本原理，从正反两方面分析高渗透率绿电园区对电网的影响：不利影响包括调峰压力、备用容量、电能质量等；有利影响包括促进消纳、降低碳排放、推动产业链等。政策建议基于定量论证（如本研究的成本数据和指标达标统计），按优先级排序给出，每条建议均有对应的数值依据支撑。

三、模型假设与符号说明

3.1 模型假设

- 1. 功率无损传输假设：**不计园区内部线路和设备的功率损耗，所有功率传输效率视为 100%，即发电端与用电端功率实时平衡。该假设简化了功率平衡方程，使问题聚焦于能量调配而非损耗计算。（来源：题目提示“不计园区功率损耗”）
- 2. 稳态运行假设：**园区运行分析时段为 1 小时，各时段内风电、光伏出力和负荷功率保持恒定，不考虑设备启停时间和爬坡约束。该假设将连续时间问题离散化为 24 时段整数规划问题，在保证工程精度的前提下显著降低了模型复杂度。（来源：题目给定分析时段为 1 小时）
- 3. 设备效率恒定假设：**电解槽制氢效率、合成氨装置产率及储能设备的充放电效率均按额定参数计算，不考虑温度、负载率等因素对效率的影响。该假设使各设备的输入输出关系简化为线性比例关系，便于采用线性规划方法求解。（来源：题目给定额定参数）
- 4. 成本构成简化假设：**风光发电设备的度电成本已包含其投资折旧和运行维护成本，电解槽和合成氨装置的运行维护成本按耗电量线性计算，不考虑设备折旧对日产氨成本的影响。该假设使吨氨成本的计算聚焦于运行成本，避免了设备投资折旧年限等

不确定性因素对结果的影响。（来源：题目给定度电成本和运维系数）

5. **场景等概率假设：**24 种风光出力场景等概率出现，每种场景代表 15 天，全年按 360 天计。该假设将无限多的实际天气模式归纳为有限个典型场景，在计算可行性与统计代表性之间取得平衡。（来源：题目给定 6 种风电场景和 4 种光伏场景的组合）

3.2 符号说明

本文使用的主要符号及其含义如表1所示，其他变量在文中首次出现时另行说明。

表 1 主要符号说明

符号	单位	含义
t	h	时段索引， $t = 0, 1, \dots, 23$
P_w	MW	风电装机容量（40 MW）
P_{pv}	MW	光伏装机容量（64 MW）
P_L	MW	常规电负荷（峰值 6 MW）
P_{prod}	MW	制氢氨装置额定功率
P_b	MW	网购电力功率
P_s	MW	售电（上网）功率
C_b	元/MWh	分时购电价（峰 802.4/平 607.4/谷 342.4）
C_s	元/MWh	上网电价（377.9）
Q_{NH_3}	t/d	日制氨产量
E_{re}	MWh	新能源发电量
E_{total}	MWh	总用电量
E_{sell}	MWh	上网电量
E_{buy}	MWh	网购电量
R_{sc}	—	新能源自发自用比例（要求 $\geq 60\%$ ）
R_g	—	总用电绿电比例（要求 $\geq 30\%$ ）
R_{cur}	—	新能源上网电量比例（要求 $\leq 20\%$ ）

四、问题一模型的建立与求解

4.1 模型建立

问题一给定典型日风光出力曲线和常规负荷曲线，电解槽与合成氨装置满负荷连续运行（36 吨/日，20.75 MW），不计功率损耗。这是一个确定性的功率平衡核算问题，通过解析计算即可得到各时段的购售电功率及全日累计指标。

第一步：功率平衡。园区内部所有发电功率与用电功率必须实时平衡。以小时为时段单位， $t = 0, \dots, 23$ 。发电侧包括风电 $P_w(t)$ 、光伏 $P_{pv}(t)$ 和网购电力 $P_b(t)$ ；用电侧包括常规负荷 $P_L(t)$ 、制氢氨负荷 $P_{prod}(t)$ 和售电 $P_s(t)$ 。功率平衡方程为：

$$P_w(t) + P_{pv}(t) + P_b(t) = P_L(t) + P_{prod}(t) + P_s(t) \quad (1)$$

各功率分量由附件给出的标么值乘以对应装机容量计算得到。风电装机 40 MW、光伏装机 64 MW、常规负荷峰值 6 MW，制氢氨在满发条件下恒为 20.75 MW：

$$P_w(t) = 40 \times \text{风电标么值}(t), P_{pv}(t) = 64 \times \text{光伏标么值}(t)$$

$$P_L(t) = 6 \times \text{负荷标么值}(t), P_{prod}(t) \equiv 20.75$$

购电和售电在任一时段不能同时发生，因此需要根据供需差额判定。引入净负荷 $N(t)$ 表示风光出力减去常规负荷和制氢氨负荷后的剩余：

$$N(t) = P_w(t) + P_{pv}(t) - P_L(t) - P_{prod}(t) \quad (2)$$

当 $N(t) \geq 0$ 时，风光出力有盈余，余电上网 $P_s(t) = N(t)$ ；当 $N(t) < 0$ 时，出力不足，需网购电力 $P_b(t) = -N(t)$ ：

$$P_s(t) = \max\{N(t), 0\}, P_b(t) = \max\{-N(t), 0\} \quad (3)$$

第二步：全日累计电量。计算出各时段购售电功率后，需汇总全日总量以计算绿电指标。全日新能源发电量 E_{re} 、总用电量 E_{total} 、上网电量 E_{sell} 和网购电量 E_{buy} 分别为各时段对应功率的累加：

$$E_{re} = \sum_{t=0}^{23} [P_w(t) + P_{pv}(t)] \quad (4)$$

$$E_{total} = \sum_{t=0}^{23} [P_L(t) + P_{prod}(t)] \quad (5)$$

$$E_{sell} = \sum_{t=0}^{23} P_s(t), E_{buy} = \sum_{t=0}^{23} P_b(t) \quad (6)$$

第三步：绿电直连指标。根据国家绿电直连项目要求，需考核以下三项指标。发自自用比例 R_{sc} 衡量新能源发电量中园区自用的比例，总用电绿电比例 R_g 衡量用电量中绿电的占比，上网电量比例 R_{cur} 衡量新能源发电量中上网的比例：

$$R_{sc} = \frac{E_{total} - E_{sell} - E_{buy}}{E_{re}} \geq 60\% \quad (7)$$

$$R_g = \frac{E_{re} - E_{sell}}{E_{total}} \geq 30\% \quad (8)$$

$$R_{cur} = \frac{E_{sell}}{E_{re}} \leq 20\% \quad (9)$$

第四步：吨氨成本。吨氨成本是评价园区经济效益的核心指标，由发电成本、购售电净成本和运维成本三部分构成：

$$C_{\text{NH}_3} = \frac{C_{\text{gen}} + C_{\text{grid}} + C_{\text{OM}}}{Q_{\text{NH}_3}} \quad (10)$$

其中发电成本按风电度电成本 0.15 元/kWh 和光伏度电成本 0.12 元/kWh 计算，购售电净成本反映园区与电网交易的结果（正为净支出、负为净收入），运维成本按设备运行时间线性计算：

$$C_{\text{gen}} = \sum_{t=0}^{23} [P_w(t) \times 150 + P_{\text{pv}}(t) \times 120] \quad [\text{元}] \quad (11)$$

$$C_{\text{grid}} = \sum_{t=0}^{23} [P_b(t)C_b(t) - P_s(t)C_s] \quad [\text{元}] \quad (12)$$

$$C_{\text{OM}} = \sum_{t=0}^{23} [20 \times 100 + 20 \times 150 + 1.5 \times 2] \quad [\text{元}] \quad (13)$$

其中 $C_b(t)$ 为分时购电价（高峰时段 10:00-15:00、18:00-21:00 为 0.8024 元/kWh，平时段为 0.6074 元/kWh，低谷时段 23:00-7:00 为 0.3424 元/kWh）， $C_s = 0.3779$ 元/kWh 为上网电价， $Q_{\text{NH}_3} = 36$ 吨为日制氨产量。

4.2 模型求解

步骤一：数据准备。从附件 1 读取典型日负荷标么值 $L_{\text{p.u.}}(t)$ ，从附件 2 读取风电标么值 $W_{\text{p.u.}}(t)$ 和光伏标么值 $S_{\text{p.u.}}(t)$ 。乘以对应装机容量得到逐时功率序列：

$$P_w(t) = 40 \times W_{\text{p.u.}}(t), \quad P_{\text{pv}}(t) = 64 \times S_{\text{p.u.}}(t), \quad P_L(t) = 6 \times L_{\text{p.u.}}(t), \quad P_{\text{prod}}(t) \equiv 20.75$$

步骤二：逐时功率平衡。对 $t = 0$ 至 23，计算净负荷：

$$N(t) = P_w(t) + P_{\text{pv}}(t) - P_L(t) - P_{\text{prod}}(t)$$

若 $N(t) \geq 0$ 则余电上网， $P_s(t) = N(t)$ ， $P_b(t) = 0$ ；若 $N(t) < 0$ 则网购电力， $P_s(t) = 0$ ， $P_b(t) = -N(t)$ 。

步骤三：全日累计。将 24 个时段的功率求和：

$$\begin{aligned} E_{\text{re}} &= \sum_{t=0}^{23} [P_w(t) + P_{\text{pv}}(t)] = 603.45 \text{ MWh} \\ E_{\text{total}} &= \sum_{t=0}^{23} [P_L(t) + P_{\text{prod}}(t)] = 558.72 \text{ MWh} \\ E_{\text{sell}} &= \sum_{t=0}^{23} P_s(t) = 216.77 \text{ MWh}, \quad E_{\text{buy}} = \sum_{t=0}^{23} P_b(t) = 172.04 \text{ MWh} \end{aligned}$$

步骤四：指标计算。将累计电量代入绿电指标公式：

$$R_{sc} = \frac{558.72 - 216.77 - 172.04}{603.45} = \frac{169.91}{603.45} = 28.16\% \quad (\text{要求} \geq 60\%, \text{不达标})$$

$$R_g = \frac{603.45 - 216.77}{558.72} = \frac{386.68}{558.72} = 69.21\% \quad (\text{要求} \geq 30\%, \text{达标})$$

$$R_{cur} = \frac{216.77}{603.45} = 35.92\% \quad (\text{要求} \leq 20\%, \text{不达标})$$

步骤五：成本计算。

$$C_{gen} = \sum P_w \times 150 + \sum P_{pv} \times 120 = 245.05 \times 150 + 358.40 \times 120 = 79\,765 \text{ 元}$$

$$C_{grid} = \sum P_b C_b - \sum P_s C_s = 15\,803 \text{ 元} \quad (\text{净购电支出})$$

$$C_{OM} = 24 \times (20 \times 100 + 20 \times 150 + 1.5 \times 2) = 60\,036 \text{ 元}$$

$$C_{NH_3} = \frac{79\,765 + 15\,803 + 60\,036}{36} = 4\,322.34 \text{ 元/吨}$$

根据上述步骤得到的 24 个时段功率数据，绘制功率平衡曲线如图 1 所示。

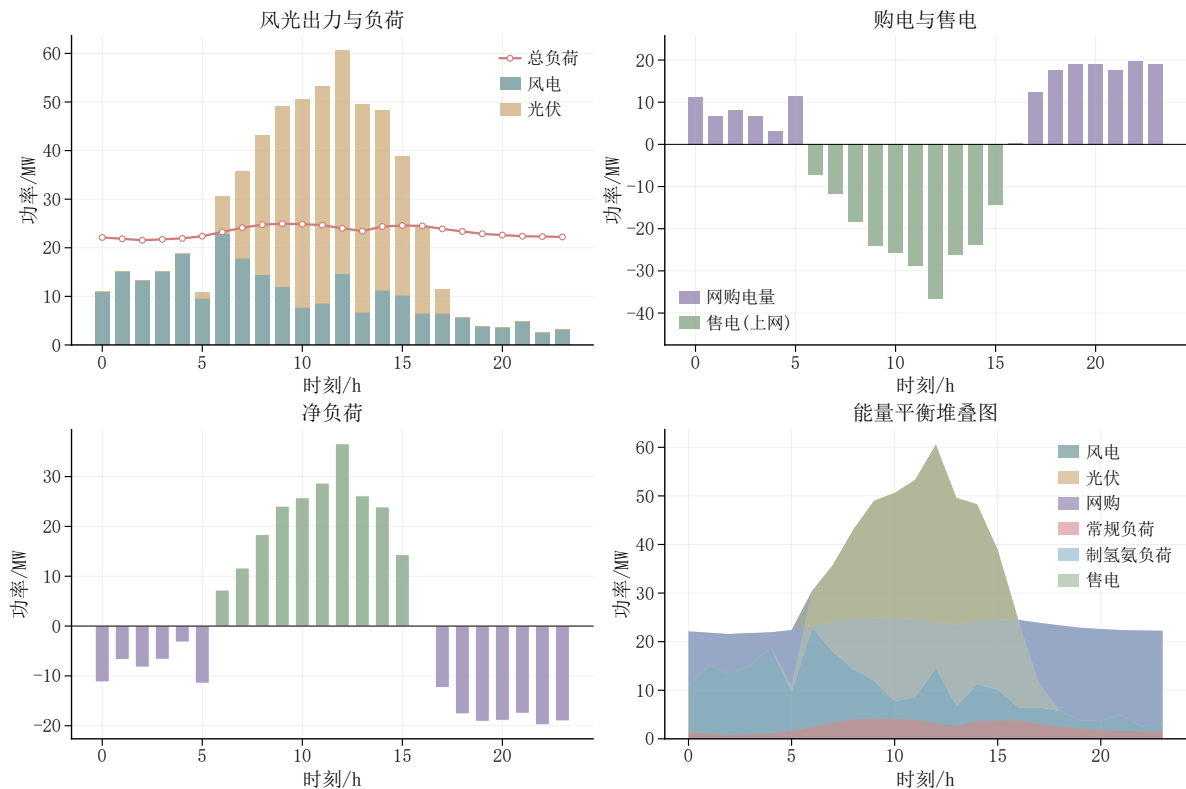


图 1 园区典型日功率平衡曲线

图 1 从四个维度展示了功率平衡情况：(a) 为风电、光伏出力和总负荷的对比，光伏在 10:00-14:00 出力最大（峰值约 46 MW），风电在凌晨和傍晚相对较强（峰值约 23 MW）；(b) 为购电和售电功率，可见夜间和午间有余电上网，傍晚负荷高峰且光伏归零

时需网购电力；(c) 为净负荷曲线，正值为余电、负值为缺电，清晰反映了购售电交替规律；(d) 为能量平衡堆叠图。

由步骤三和步骤四得到全日能量指标和绿电指标，汇总于表 2。

表 2 问题一：绿电直连指标与成本

指标	计算值	要求
新能源发电量 E_{re}	603.45 MWh	—
总用电量 E_{total}	558.72 MWh	—
上网电量 E_{sell}	216.77 MWh	—
网购电量 E_{buy}	172.04 MWh	—
自发自用比例 R_{sc}	28.16%	$\geq 60\%$ ×
总用电绿电比例 R_g	69.21%	$\geq 30\%$ ✓
上网电量比例 R_{cur}	35.92%	$\leq 20\%$ ×
吨氨成本 C_{NH_3}	4322.34 元/吨	—

表 2和图 2使用雷达图直观展示了三个指标与要求值的差距。

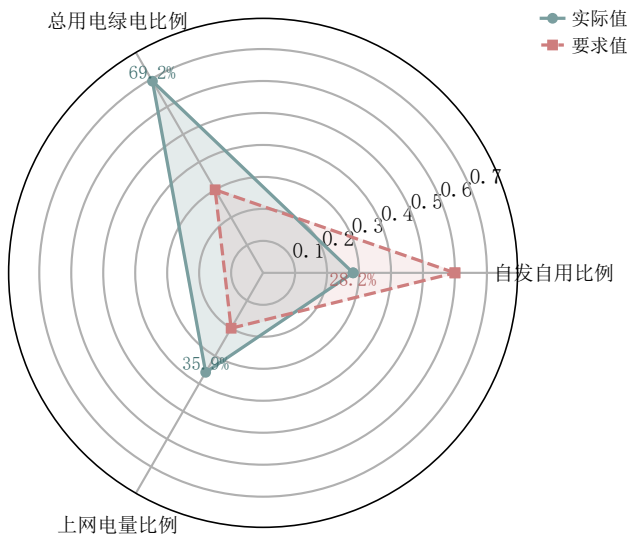


图 2 绿电指标达标雷达图

原因分析：三个指标中仅有总用电绿电比例达标（69.21%），自发自用比例（28.16%）和上网电量比例（35.92%）不达标。园区新能源总装机容量 104 MW（风电 40 MW+ 光伏 64 MW），而制氢氨负荷仅 20.75 MW（对应 36 吨/日产能），总负荷峰值约 25 MW。新能源发电能力远超自用需求，大量绿电上网，导致自发自用比例偏低（仅 169.91 MWh 自用，占发电量的 28.16%）和上网比例偏高（35.92%）。这一结果表明，36 吨/日产能下

制氢氨负荷相对于风光装机容量过小，需要通过提升产能或调整可再生能源装机规模来满足绿电直连指标要求 [5]。

五、问题二模型的建立与求解

5.1 模型建立

问题二中园区制氨产能增至 72 吨/日，制氢氨设备功率按比例线性提升至 41.5 MW (ALKEL 20 MW + PEMEL 20 MW + 合成氨装置 1.5 MW，产率 3.0 t/h)。电氢氨装置只有全额开机和停机两种运行方式，制氨产量从 72 吨/日起按 9 吨/日递减至 36 吨/日。这是一个典型的 0-1 整数规划问题。

定义 0-1 变量 $x(t) \in \{0, 1\}$ 表示时段 t 是否开机生产：

$$x(t) = \begin{cases} 1, & \text{时段}t\text{开机生产} \\ 0, & \text{时段}t\text{停机} \end{cases} \quad t = 0, 1, \dots, 23 \quad (14)$$

目标为最小化日净电费（购电成本减去售电收益）与运维成本之和：

$$\min F = \sum_{t=0}^{23} [P_b(t)C_b(t) - P_s(t)C_s + C_{OM}(t)] \quad (15)$$

当 $x(t) = 1$ 时， $P_{\text{prod}}(t) = P_{\text{max}} = 41.5 \text{ MW}$ ；当 $x(t) = 0$ 时， $P_{\text{prod}}(t) = 0$ 。运维成本 $C_{OM}(t) = x(t) \times 5003 \text{ 元}$ 。

约束条件包括功率平衡约束、日产量约束和购售电互斥约束：

$$P_w(t) + P_{pv}(t) + P_b(t) = P_L(t) + x(t)P_{\text{max}} + P_s(t) \quad \forall t \quad (16)$$

$$\sum_{t=0}^{23} x(t) = k, \quad k = Q_{\text{NH}_3}/3 \in \{24, 21, 18, 15, 12\} \quad (17)$$

$$P_b(t) \geq 0, P_s(t) \geq 0, P_b(t) \cdot P_s(t) = 0 \quad \forall t \quad (18)$$

整理为标准整数规划形式：

$$\begin{aligned} \min \quad & \sum_{t=0}^{23} [C_b(t) \max\{P_L(t) + x(t)P_{\text{max}} - P_w(t) - P_{pv}(t), 0\} \\ & - C_s \max\{P_w(t) + P_{pv}(t) - P_L(t) - x(t)P_{\text{max}}, 0\} + 5003x(t)] \\ \text{s.t.} \quad & \sum_{t=0}^{23} x(t) = k, \quad x(t) \in \{0, 1\}, \quad \forall t \end{aligned} \quad (19)$$

5.2 模型求解

步骤一：确定开机时长。根据目标日产量 Q_{NH_3} ，计算所需开机小时数 $k = Q_{\text{NH}_3}/3$ 。72、63、54、45、36 吨/日分别对应 $k = 24, 21, 18, 15, 12$ 小时。

步骤二：计算各时段边际成本。对于每个时段 $t = 0, \dots, 23$ ，按以下公式计算相对于不开机而言，开机的边际成本 $\Delta C(t)$ ：

$$\Delta C(t) = [C_{\text{on}}(t) - C_{\text{off}}(t)] + 5003 \quad (20)$$

其中：

$$C_{\text{off}}(t) = \begin{cases} -[P_w(t) + P_{\text{pv}}(t) - P_L(t)] \cdot C_s, & \text{若可再生过剩} \\ [P_L(t) - P_w(t) - P_{\text{pv}}(t)] \cdot C_b(t), & \text{若可再生不足} \end{cases}$$

$$C_{\text{on}}(t) = \begin{cases} -[P_w(t) + P_{\text{pv}}(t) - P_L(t) - 41.5] \cdot C_s, & \text{若开机后仍过剩} \\ [P_L(t) + 41.5 - P_w(t) - P_{\text{pv}}(t)] \cdot C_b(t), & \text{若开机后需购电} \end{cases}$$

常数 5003 元是开机一小时产生的运维费用（ALKEL 20MW $\times$ 100 + PEMEL 20MW $\times$ 150 + NH₃ 1.5MW $\times$ 2）。

以 $t = 12$ （12:00-13:00，峰电价 0.8024 元/kWh）为例：

$$C_{\text{off}}(12) = -(14.52 + 46.08 - 3.26) \times 377.9 = -21\,668 \text{ 元} \quad (\text{售电收益})$$

$$C_{\text{on}}(12) = -(14.52 + 46.08 - 3.26 - 41.5) \times 377.9 = -5\,985 \text{ 元}$$

$$\Delta C(12) = (-5\,985) - (-21\,668) + 5\,003 = -10\,680 \text{ 元}$$

边际成本为负，说明 $t = 12$ 开机比停机更省钱（将低价值余电转为氨生产，扣除运维后仍有净收益）。

以 $t = 19$ （19:00-20:00，峰电价 0.8024 元/kWh）为例：

$$C_{\text{off}}(19) = (2.12 - 3.69) \times 0 = 0 \text{ 元} \quad (\text{无可再生能源，但净负荷为负})$$

$$C_{\text{on}}(19) = (2.12 + 41.5 - 3.69) \times 802.4 = 32\,088 \text{ 元}$$

$$\Delta C(19) = 32\,088 - 0 + 5\,003 = 37\,091 \text{ 元}$$

边际成本很高，说明 $t = 19$ 开机需大量购电，成本极高。

步骤三：选择最优时段。将 24 个 $\Delta C(t)$ 从小到大排序，选择前 k 个最小的时段开机。以 36 吨/日（ $k = 12$ ）为例，边际成本最低的 12 个时段集中在 10:00-21:00（太阳能充足或风能较好的时段），而夜间和清晨因光伏为零且风电不够支撑满负荷，边际成本较高，因此不开机。

步骤四：计算指标和成本。根据选定的开机方案，计算购售电功率，然后按模型建立部分的公式计算绿电指标和吨氨成本。各产量结果如表 3 所示。

表 3 典型场景各产量运行指标

产量 (t/d)	时长 (h)	成本 (元/吨)	自用比例	绿电比例	上网比例	达标
72	24	6219	86.5%	53.3%	6.7%	✓
63	21	5328	84.3%	59.7%	7.8%	✓
54	18	4591	80.2%	67.3%	9.9%	✓
45	15	3945	51.0%	66.7%	24.5%	×
36	12	3192	10.5%	59.7%	44.7%	×

表 3展示了各产量下的成本与绿电指标。随着产量降低，运行小时数减少，购电量也随之下降，使得吨氨成本从 72 t/d 的 6219 元/吨降至 36 t/d 的 3192 元/吨。然而，低产量下自发自用比例也大幅下滑——36 t/d 时仅 10.5%（不满足 60% 要求），而上网比例达到 44.7%（不满足 20% 要求），说明离散调节下低产工况难以兼顾经济性与绿电指标。

为进一步分析开机策略，根据各产量的最优开机时段绘制甘特图如图 3所示。

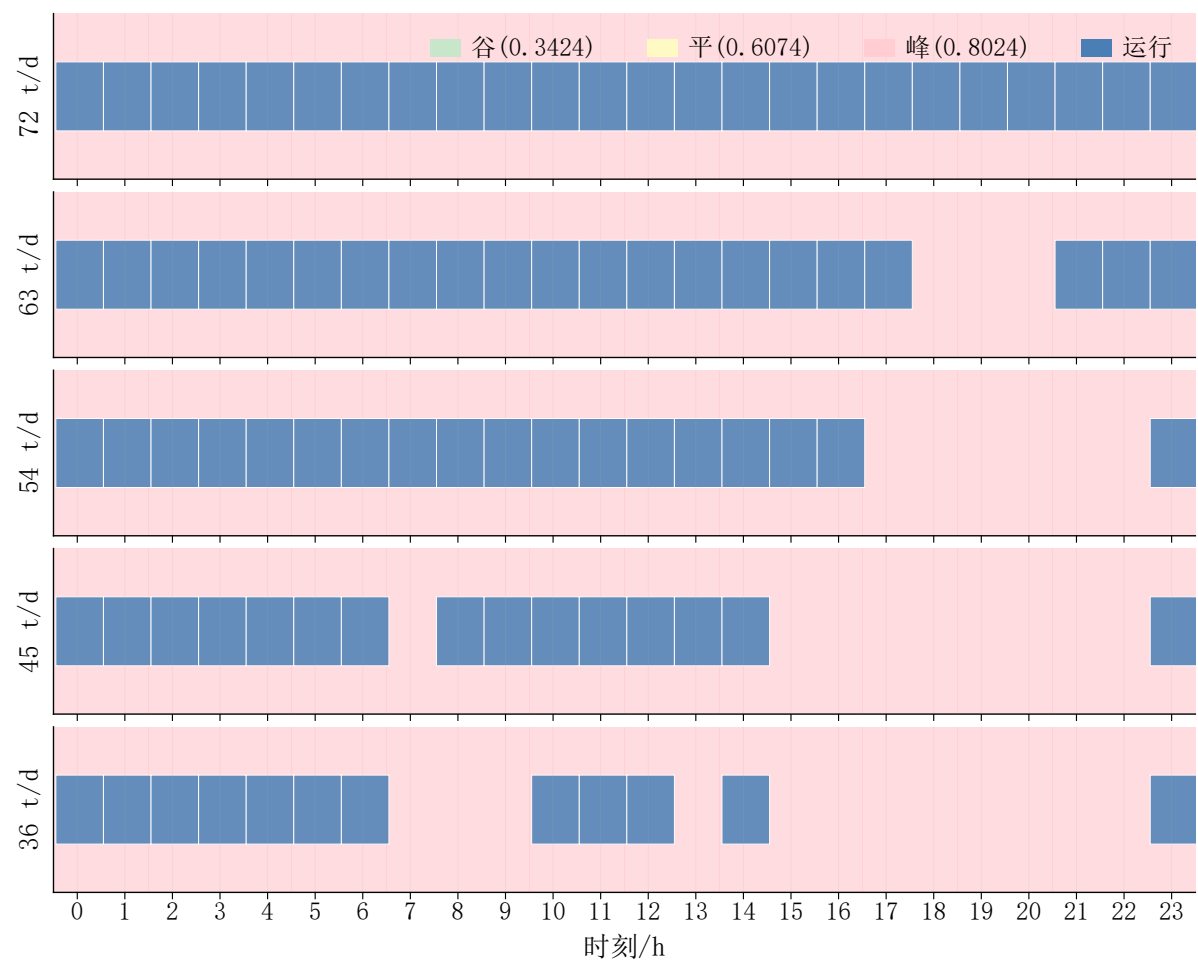


图 3 各产量最优生产时段安排

图 3展示了各产量下最优的 24 小时开机方案（蓝色条）与电价时段（谷时段绿色、平时段黄色、峰时段粉红）的关系。72 t/d 需满发 24h，无法选择；63 t/d 优先关停了 18:00-20:00 峰时段（此时光伏归零而负荷攀升，购电成本高达 0.8024 元/kWh）；54 t/d、45 t/d、36 t/d 依次继续关停成本最高的时段，优先保留谷电价时段（0.3424 元/kWh）和光伏大发的中午时段。这表明边际成本排序算法倾向于在低价时段和高可再生时段安排生产，有效降低了购电成本。

由各产量的成本和绿电指标数据得到图 4。

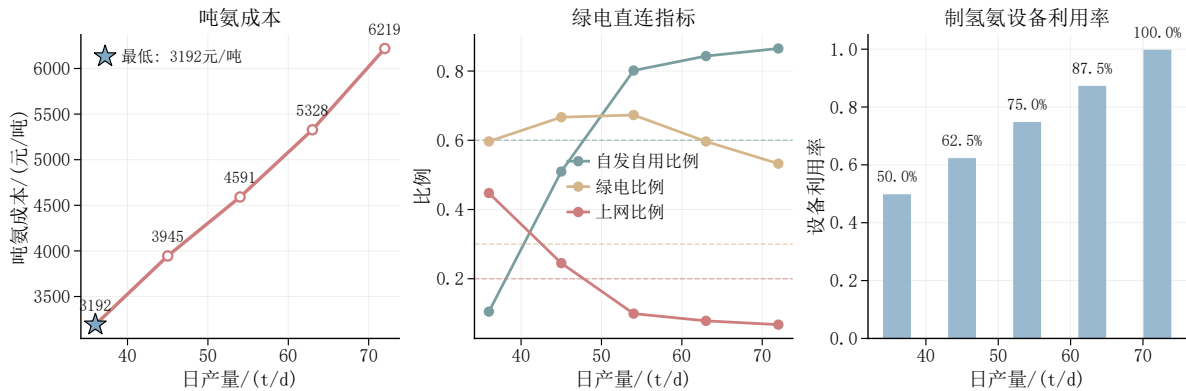


图 4 关键指标随产量变化

图 4包含三个子图。(a) 吨氨成本曲线：从 72 t/d 的 6,219 元/吨单调下降至 36 t/d 的 3,192 元/吨，降幅约 49%。这是因为低产量下设备运行时间短，购电费用减少、售电收益增加，单位成本被摊薄。(b) 绿电指标曲线：自用比例在 72-54 t/d 时稳定在 80% 以上，但 45 t/d 时骤降至 51.0%（低于 60% 要求），36 t/d 时进一步降至 10.5%。上网比例在 54 t/d 时仅 9.9%（达标），45 t/d 时升至 24.5%（超标）。这表明离散调节下低产量时大部分可再生电力被上网出售而非自用，导致指标恶化。(c) 设备利用率：随产量线性降低，从 100% 降至 50%。

以上分析均基于典型日风光出力场景。然而园区实际运行中风光出力具有显著的随机波动性——风电可能全天低于额定功率的 10%，光伏可能因多云天气出力减半。为全面评估离散调节方案在不同天气条件下的表现，将附件 3 的 6 种风电场景与附件 4 的 4 种光伏场景进行两两组合，共得到 24 种风光出力场景（如 S1 风 1 光 1、S2 风 1 光 2、…、S24 风 6 光 4）。对每种场景重复上述边际成本优化，计算各产量水平下的吨氨成本，绘制热力图如图 5 所示。

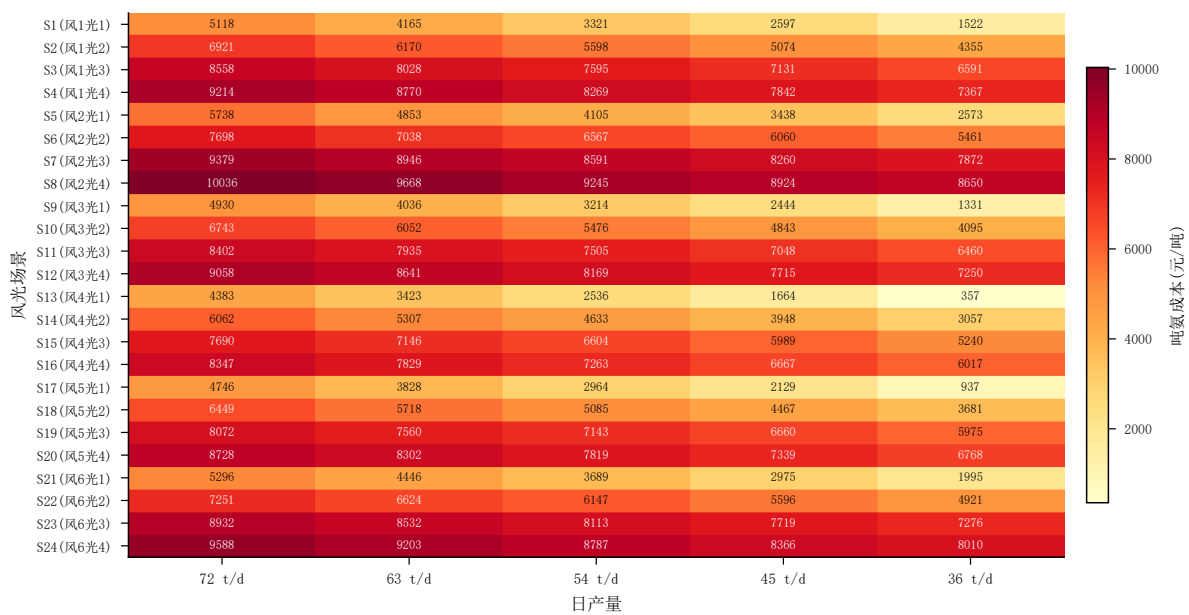


图 5 24 场景 × 5 产量吨氨成本热力图

图 5 中每一行代表一种风光场景，每一列代表一个产量水平，颜色深浅反映吨氨成本高低。场景 S8（风 2 光 4，风电和光伏均偏低）在所有产量下成本均为最高，原因是该场景可再生发电总量少，需大量网购高价电力；场景 S13（风 4 光 1，风电高光伏中等）成本最低，因为风电在夜间和全天持续出力。高产量时各场景间成本差异较小（约 1,000-2,000 元/吨），低产量时差异显著增大（可达 5,000 元/吨以上），说明低产量下风光条件对经济性的影响更为敏感。

各产量下 24 种场景的绿电指标达标情况统计如图 6 所示。

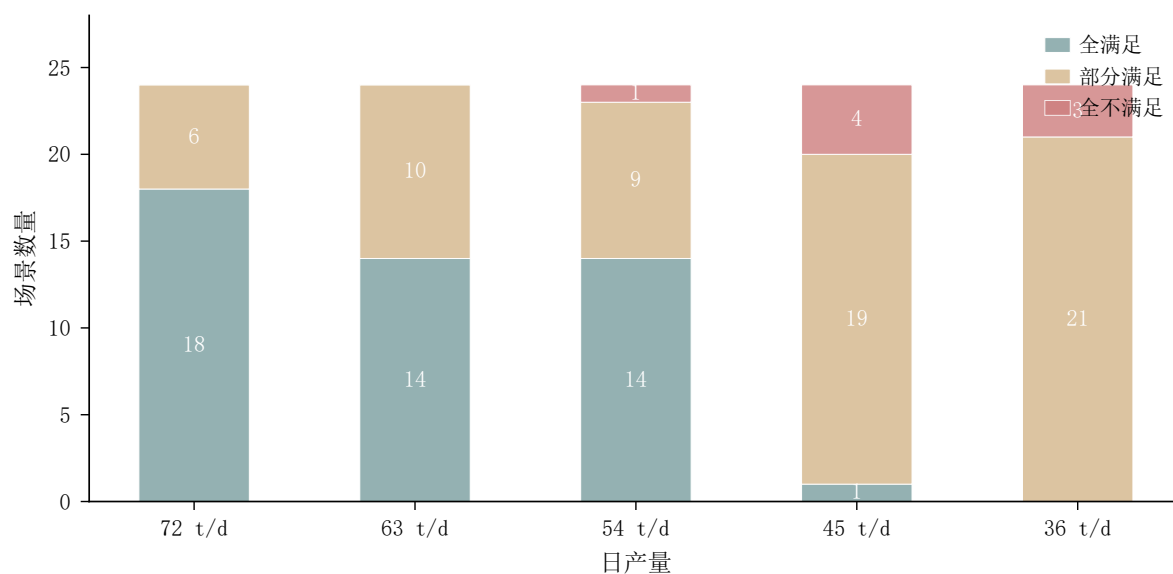


图 6 各产量绿电指标达标统计

图 6 按全满足（绿）、部分满足（黄）、全不满足（红）三类统计了 24 场景的达标情况。72 t/d 时 18 个场景全满足、6 个部分满足，表现最好；63 t/d 和 54 t/d 分别有 14 个全满足，仍属良好；45 t/d 时仅 1 个全满足、19 个部分满足、4 个全不满足，急剧恶化；36 t/d 时 0 个全满足、21 个部分满足、3 个全不满足。这说明离散调节下产量降至 45 t/d 及以下时，即使针对不同风光场景优化调度，也无法使大多数场景的绿电指标达标。根本原因是开机时长过短导致可再生自用电量太少、上网比例过高。

六、问题三模型的建立与求解

6.1 模型建立

问题三将电氢氨装置的运行方式从离散开关改为功率连续可调（下限为额定功率的 10%）。各时段的制氢氨功率 $p(t)$ 可在 $[0.1P_{\max}, P_{\max}]$ 区间内连续调节，也可完全停机 ($p(t) = 0$)。这是一个连续线性规划问题。

定义连续变量 $p(t)$ 为时段 t 的制氢氨功率：

$$p(t) \in \{0\} \cup [0.1P_{\max}, P_{\max}], \quad t = 0, 1, \dots, 23 \quad (21)$$

其中 $P_{\max} = 41.5 \text{ MW}$ 。

目标为最小化日净电费与运维成本之和：

$$\min F = \sum_{t=0}^{23} [P_b(t)C_b(t) - P_s(t)C_s + C_{\text{OM}}(t)] \quad (22)$$

运维成本 $C_{\text{OM}}(t) = p(t) \times 126 \text{ 元}$ 。

约束条件包括功率平衡、功率上下限和日产量约束：

$$P_w(t) + P_{pv}(t) + P_b(t) = P_L(t) + p(t) + P_s(t) \quad \forall t \quad (23)$$

$$p(t) \in \{0\} \cup [0.1P_{\max}, P_{\max}] \quad \forall t \quad (24)$$

$$\sum_{t=0}^{23} p(t) \cdot \eta = Q_{\text{NH}_3}, \quad \eta = \frac{3.0}{41.5} = 0.0723 \text{ [t/MWh]} \quad (25)$$

整理为标准形式：

$$\begin{aligned} \min \quad & \sum_{t=0}^{23} \left[C_b(t) \max\{P_L(t) + p(t) - P_w(t) - P_{pv}(t), 0\} \right. \\ & \left. - C_s \max\{P_w(t) + P_{pv}(t) - P_L(t) - p(t), 0\} + 126p(t) \right] \end{aligned} \quad (26)$$

$$\text{s.t.} \quad \sum_{t=0}^{23} p(t) \cdot \eta = Q_{\text{NH}_3}, \quad p(t) \in \{0\} \cup [0.1P_{\max}, P_{\max}], \quad \forall t$$

6.2 模型求解

步骤一：计算各时段可再生盈余。对每个时段 t ，计算风光出力扣除常规负荷后的剩余功率：

$$\Delta P(t) = \max\{0, P_w(t) + P_{pv}(t) - P_L(t)\}$$

这个盈余是可免费利用的可再生电力（若不利用只能以 0.3779 元/kWh 的价格上网出售）。

步骤二：优先利用盈余。将盈余最大的时段按从大到小排序，逐时段分配制氢功率 $p(t)$ 。对于每个时段，尽量利用盈余，但不超过设备最大功率 41.5 MW 且不低于最小功率 4.15 MW：

$$p(t) = \min\{\text{剩余需求}, \Delta P(t), 41.5\}$$

若 $\Delta P(t) < 4.15$ MW，则不在此时段分配功率（无法满足下限要求）。

步骤三：不足部分按边际成本分配。如果步骤二分配后仍有剩余产量需求，计算剩余时段开机的边际成本：

$$\text{边际成本} = \begin{cases} -377.9 + 126 = -251.9 \text{ 元/MWh}, & \text{该时段可再生有盈余} \\ C_b(t) + 126 \text{ 元/MWh}, & \text{该时段需购电} \end{cases}$$

其中 -377.9 是放弃售电的机会成本，+126 是运维成本。按边际成本从小到大排序，逐时段分配功率。

以 36 吨/日目标为例，先分配盈余时段：

- $t = 12$, $\Delta P = 57.3$ MW，可取满 41.5 MW
- $t = 11$, $\Delta P = 50.1$ MW，可取满 41.5 MW
- 依次分配盈余时段后，仍有少量剩余需求

然后将剩余需求量按边际成本最小的原则分配到 $t = 0 - 6$ 谷电价时段（购电价 342.4 元/MWh+126 = 468.4 元/MWh）。

步骤四：微调余量。若最后剩余不足 4.15 MW 的功率需求，追加到已有运行时段上（在其现有功率基础上增加少量出力，确保总功率不超过 41.5 MW），保证产量精确达标。

为了直观展示连续调节与离散调节在调度策略上的本质差异，以 72 吨/日满产为例，分别运行两种优化算法得到各时段的制氢氨功率安排、购电功率和售电功率，绘制对比图如图 7 所示。其中离散调节的功率曲线呈阶梯状（要么满发 41.5 MW，要么停机 0 MW），连续调节的功率曲线可连续变化。

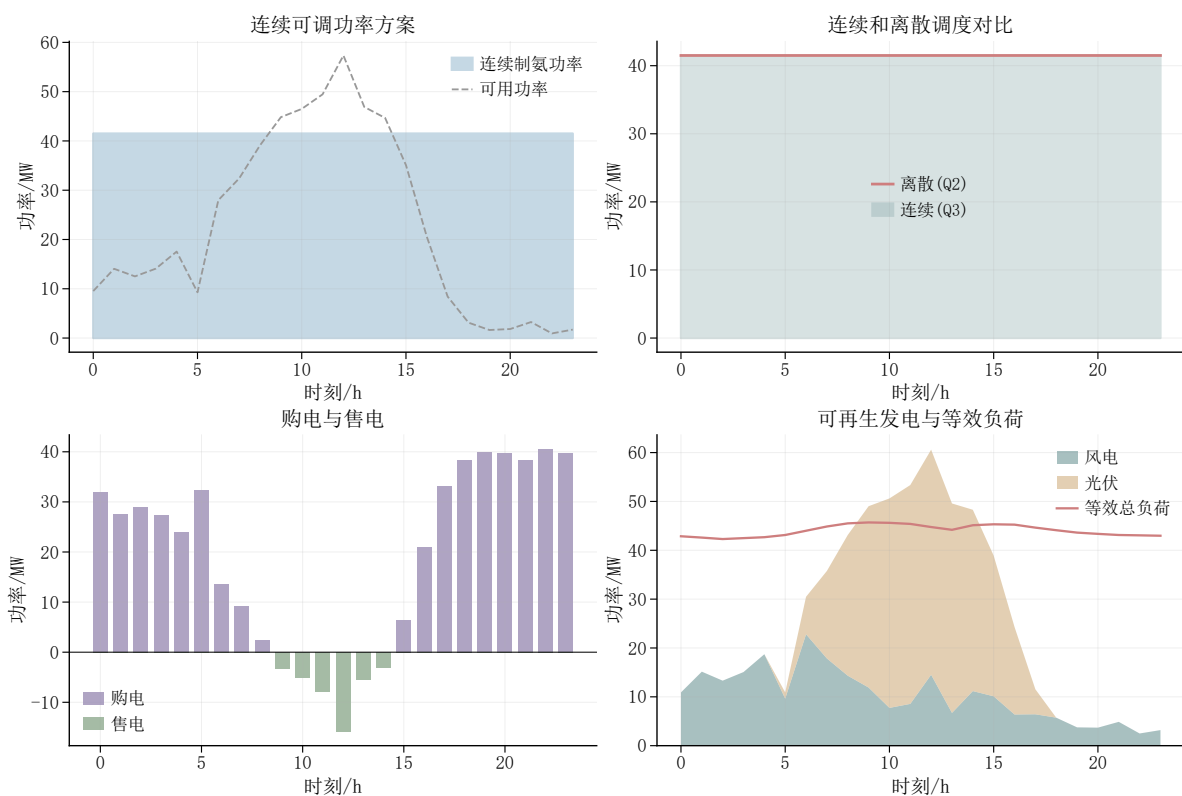


图 7 连续调节功率调度方案（72 t/d 典型场景）

图 7对比了同一场景下离散和连续调节的差异。连续调节的制氮功率平滑跟随可再生出力，避免了购售电极端波动。

各产量下 Q2 与 Q3 的关键指标对比见表 4。

表 4 Q2 与 Q3 关键指标对比（典型场景）

产量 (t/d)	吨氨成本 (元/吨)		自用比例		上网比例	
	Q2	Q3	Q2	Q3	Q2	Q3
72	6219	6219	86.5%	86.5%	6.7%	6.7%
63	5328	5858	84.3%	84.6%	7.8%	7.7%
54	4591	4758	80.2%	82.4%	9.9%	8.8%
45	3945	4005	51.0%	82.4%	24.5%	8.8%
36	3192	3406	10.5%	82.4%	44.7%	8.8%

表 4显示连续调节在各产量下自用比例稳定在 82% 以上，上网比例控制在 9% 以内，显著优于离散调节。

以上对比仅基于典型日场景，不足以全面反映连续调节的优势。为进一步验证连续调节在多种天气条件下的表现，将 6 种风电场景与 4 种光伏场景两两组合得到 24 种风

光场景，分别在离散和连续两种调节方式下进行优化。对每种场景的每个产量水平，计算其吨氨成本和绿电指标，与离散调节的结果进行逐场景对比。将各产量下 24 场景的成本均值和达标情况汇总，绘制 Q2 与 Q3 的成本对比图和达标统计对比图如图 8和图 9所示。

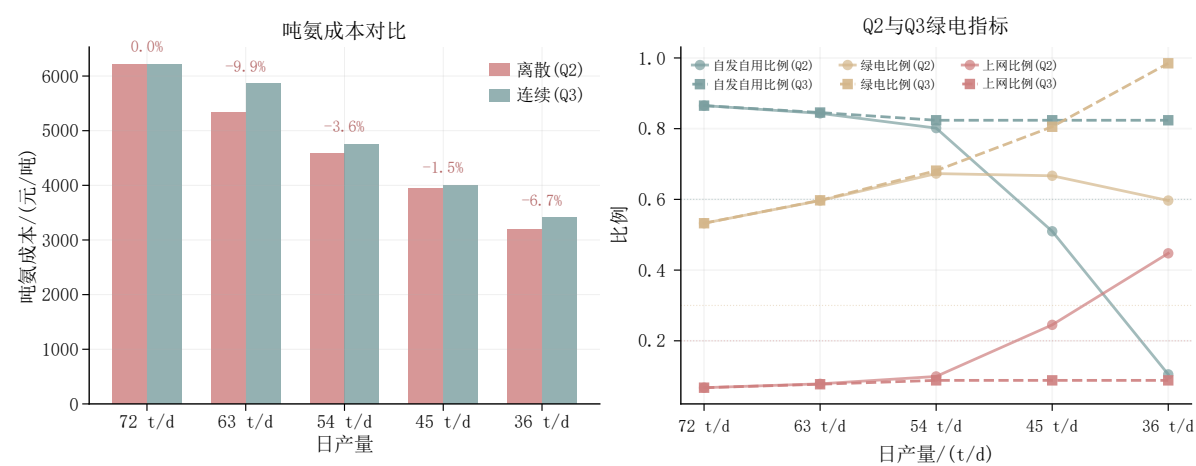


图 8 Q2 vs Q3 吨氨成本与绿电指标对比

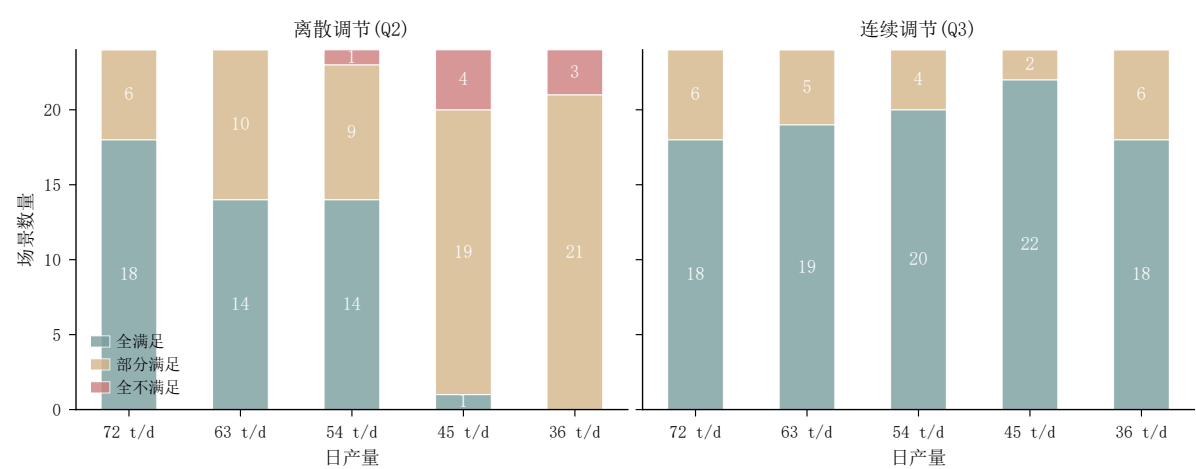


图 9 Q2 vs Q3 绿电指标达标统计对比

图 8从成本角度对比了两种调节方式。连续调节在各产量下的吨氨成本略高于离散调节（约 200-400 元/吨），这是因为连续调节不再大量出售可再生余电，而是将其用于替代网购电力，虽然增加了自发自用电量、改善了绿电指标，但也减少了售电收益。以 45 t/d 为例，离散调节下售电收益较高使得成本低至 3,945 元/吨，但自用比例仅 51.0

图 9从达标角度展示了连续调节的压倒性优势。72 t/d 时两种方式一致（均为 18 个全满足），但随着产量降低，离散调节的全满足数量急剧下降（45 t/d 仅 1 个、36 t/d 为 0 个），而连续调节的全满足数始终保持在 18-22 个的高水平。特别值得注意的是 45 t/d 这一产量——离散调节仅 1 个场景全满足，而连续调节高达 22 个。具体数量如表 5所示。

表 5 Q2 与 Q3 达标场景数对比

产量 (t/d)	全满足		部分满足		全不满足	
	Q2	Q3	Q2	Q3	Q2	Q3
72	18	18	6	6	0	0
63	14	19	10	5	0	0
54	14	20	9	4	1	0
45	1	22	19	2	4	0
36	0	18	21	6	3	0

七、问题四模型的建立与求解

7.1 模型建立

问题四要求分析园区离网运行时的能源自给状况并研究储能的最优配置。离网运行时园区与公共电网无电气连接 ($P_b(t) = P_s(t) \equiv 0$)，制氢氨功率必须跟随可再生出力变化。

离网运行下，功率平衡方程为：

$$P_w(t) + P_{pv}(t) = P_L(t) + P_{prod}(t) + P_{ch}(t) - P_{dis}(t) + P_{cur}(t) \quad (27)$$

其中 $P_{cur}(t)$ 为弃电功率。无储能时, $P_{ch}(t) = P_{dis}(t) \equiv 0$, 可用功率 $P_{avail}(t) = \max\{0, P_w(t) + P_{pv}(t) - P_L(t)\}$ ，制氢氨功率：

$$P_{prod}(t) = \begin{cases} P_{\max}, & P_{avail}(t) \geq P_{\max} \\ P_{avail}(t), & P_{\min} \leq P_{avail}(t) < P_{\max} \\ 0, & P_{avail}(t) < P_{\min} \end{cases} \quad (28)$$

其中 $P_{\min} = 0.1P_{\max} = 4.15 \text{ MW}$ 。

储能设备的荷电状态 (SOC) 动态方程 [7] 为：

$$E(t+1) = E(t)(1 - \delta) + P_{ch}(t)\eta_{ch} - \frac{P_{dis}(t)}{\eta_{dis}} \quad (29)$$

其中各参数如表 6 所示。约束条件包括 $0 \leq E(t) \leq E_{\max}$ 、充放电功率限制及充放电互斥。

表 6 储能设备参数

参数	取值
自损耗率 δ	0.2%
充电效率 η_{ch}	90%
放电效率 η_{dis}	90%
投资成本	1000 元/kWh
使用寿命	15 年

储能容量优化采用双层模型：外层枚举 $E_{\text{max}} \in [0, 200]$ MWh，目标为最小化年均吨氨成本（含储能投资折旧和运维）；内层在每种风光场景下优化逐时调度，使可再生能源利用最大、弃电最小。

7.2 模型求解

离网无储能求解。对每个场景 $t = 0, \dots, 23$ ：

1. 计算可用功率 $P_{\text{avail}}(t) = \max\{0, P_{\text{w}}(t) + P_{\text{pv}}(t) - P_{\text{L}}(t)\}$ 。
2. 若 $P_{\text{avail}}(t) \geq 41.5$ MW，取满发 $P_{\text{prod}}(t) = 41.5$ MW，弃电 $P_{\text{cur}}(t) = P_{\text{avail}}(t) - 41.5$ 。
3. 若 $4.15 \leq P_{\text{avail}}(t) < 41.5$ MW，按可用功率生产 $P_{\text{prod}}(t) = P_{\text{avail}}(t)$ ，无弃电。
4. 若 $P_{\text{avail}}(t) < 4.15$ MW，无法开机 $P_{\text{prod}}(t) = 0$ ， $P_{\text{avail}}(t)$ 全部弃掉。

由 $P_{\text{prod}}(t)$ 累计日产量： $Q_{\text{day}} = [\sum P_{\text{prod}}(t)] \times 0.0723$ 。

以 S13（风 4 光 1）为例：白天光伏大发时段（10:00-14:00）可再生盈余 50-70 MW，制氨满载 41.5 MW，多余电量弃掉；夜间和傍晚盈余不足 4.15 MW 时停机。该场景日产量约 46.2 吨，弃电 177.3 MWh。

储能配置求解。外层枚举储能容量 $E_{\text{max}} = 0, 20, 40, \dots, 200$ MWh，内层调度规则为：

1. 有盈余时：先满足生产，剩余电量优先给储能充电（不超过 SOC 上限和充电功率限制）。
2. 可再生不足时：若储能 SOC 足够，放电补充生产，使 P_{prod} 达到最小开机功率以上。

每个储能容量下计算 24 场景的年均吨氨成本（含储能折旧和运维），结果如图 12 所示。

离网联网对比求解。按题目要求，在相同产量下对比。对每个场景取离网产量 Q_{off} ，调用问题三连续调节模型计算联网模式在同等产量下的成本，然后统计全年均值。

结果分析。由最大弃电场景 S13 的功率数据绘制离网运行功率平衡图如图 10 所示。

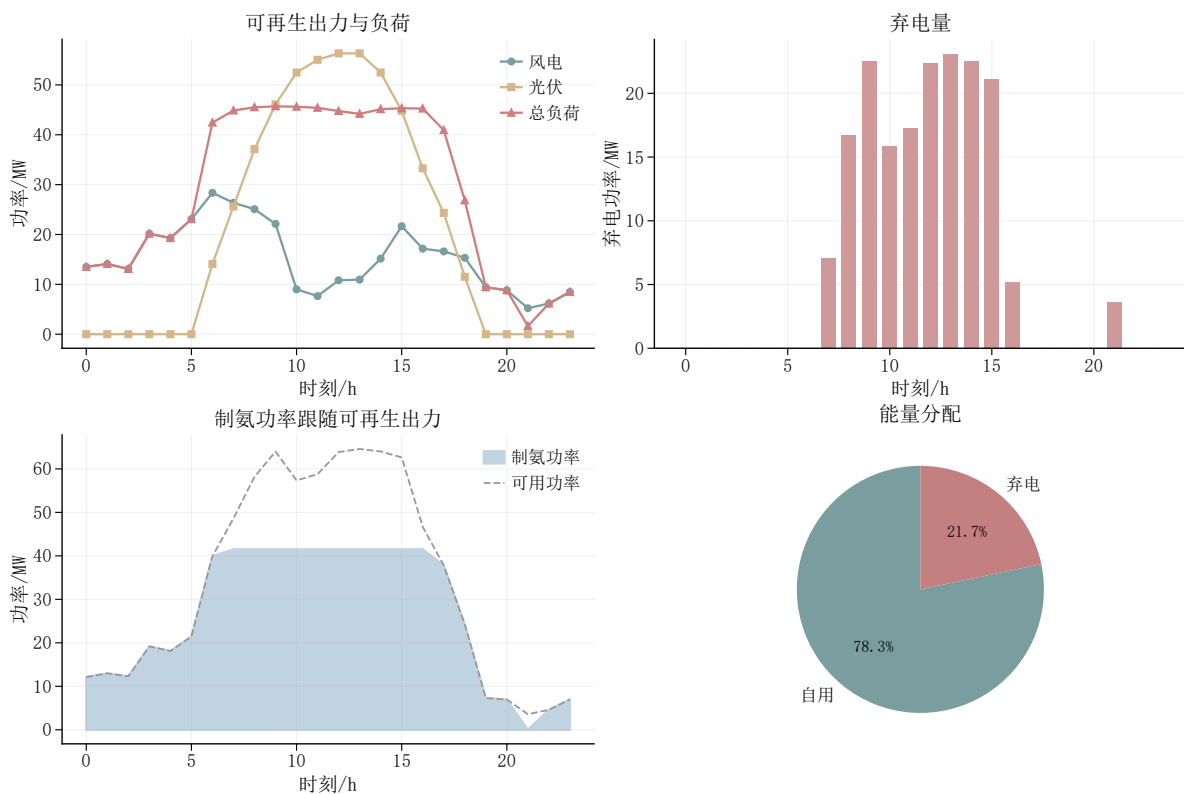


图 10 最大弃电场景（S13 风 4 光 1）离网运行功率平衡

图 10展示了离网运行时制氨功率紧密跟随可再生出力——白天光伏大发时段制氨功率接近满载，夜间和低出力时段则下降或停机。

24 场景的离网产量与弃电率分布如图 11所示，汇总指标见表 7。

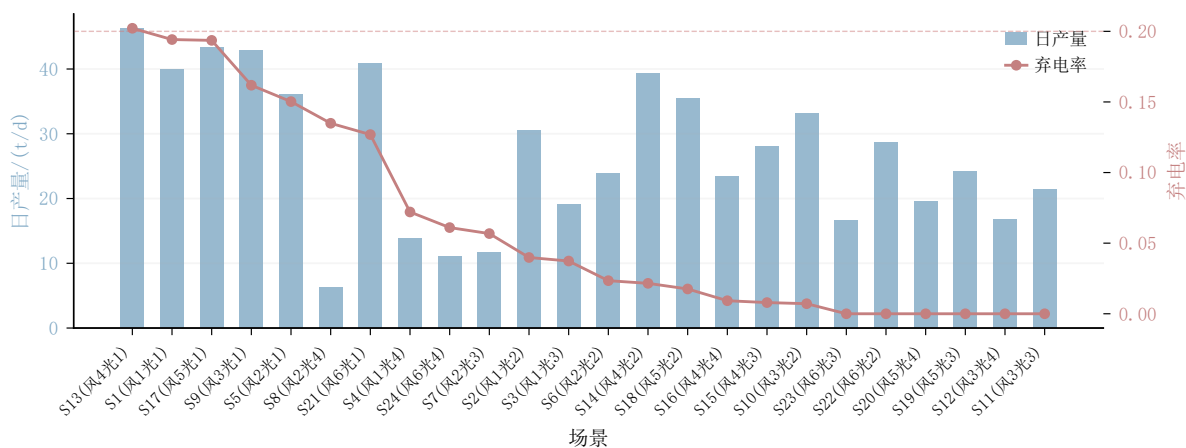


图 11 各场景产量与弃电率分析

表 7 离网无储能运行关键指标

指标	数值
全年制氨总量	9782 吨
平均吨氨成本	4158 元/吨
平均产能利用率	37.7%
平均可再生利用率	93.7%
最大弃电场景	S13(风 4 光 1): 177.3 MWh
最小产量场景	S8(风 2 光 4): 6.2 t/d

图 11 以场景按弃电率降序排列的方式展示了产量与弃电率的关系。总体趋势是弃电率越高的场景产量也越高，这是因为高弃电场景通常风光资源丰富（如 S13 风 4 光 1），可再生出力远超制氢负荷需求，制氨功率可长时间满载运行，多余电量则被弃掉。反之，低弃电场景（如 S8 风 2 光 4）风光出力偏低，可用功率经常低于最小开机阈值 4.15 MW，导致产量极低但弃电也少。表 7 汇总了 24 场景的总体情况：全年制氨 9,782 吨、平均产能利用率仅 37.7%，说明离网模式下大部分时间可再生出力不足以支撑满负荷生产。

储能配置优化的 Pareto 曲线如图 12 所示。

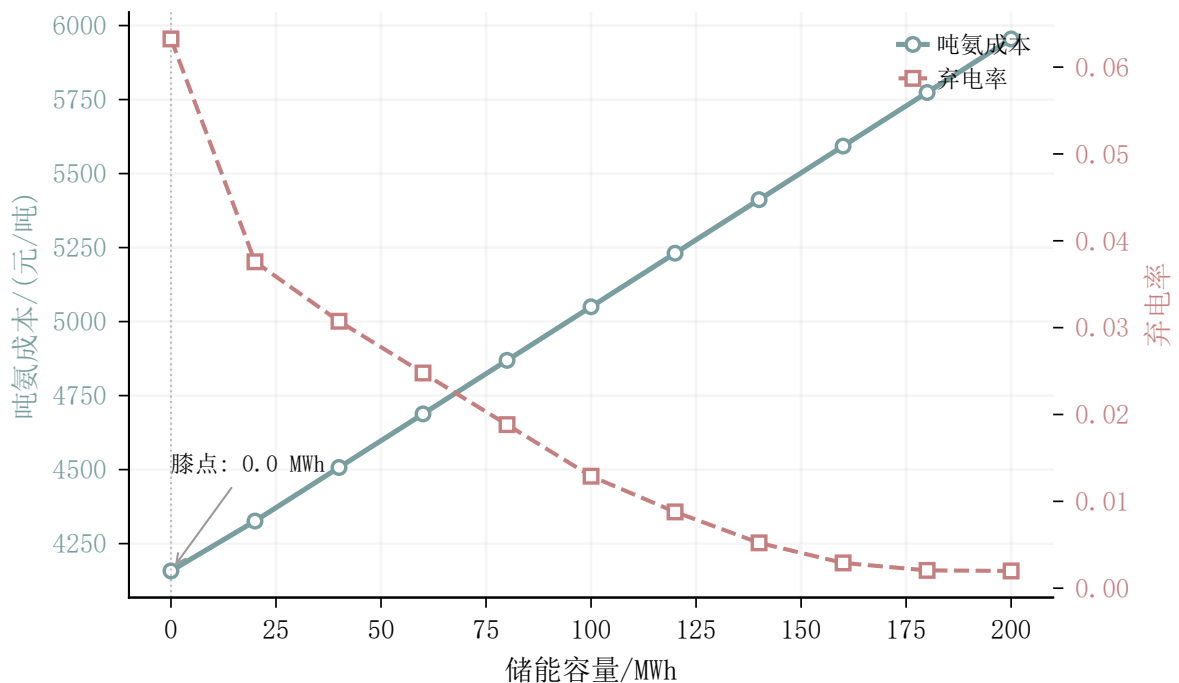


图 12 储能容量优化 Pareto 曲线

图 12 中横轴为储能容量（0-200 MWh），左右纵轴分别为平均吨氨成本和弃电率。

曲线显示随着储能容量增加，弃电率从 93.7% 缓慢下降，但吨氨成本因储能投资折旧而持续上升。转折点出现在 0 MWh 处——配备储能后吨氨成本立即升高，而弃电率改善微乎其微（20 MWh 储能仅将可再生利用率从 93.7% 提升至 93.9%）。这是因为限制离网产量的根本因素是能量总量而非功率波动：即使配备储能，低风光日的总发电量也不足以满足满负荷生产需求，储能仅在少数盈余时段充放电，边际效益远低于投资成本。因此在当前储能成本（1,000 元/kWh）下最优容量为 0 MWh。离网与联网模式在相同产量下的对比见表 8。

表 8 离网与联网模式综合对比（相同产量）

指标	离网	联网 (同等产量)
吨氨成本 (元/吨)	4158	3697
年产量 (吨)	9782	9759
产能利用率	37.7%	37.6%

表 8表明在年产量基本持平（离网 9,782 吨 vs 联网 9,759 吨，差异仅 0.2%）的条件下，联网模式的吨氨成本为 3,697 元/吨，比离网模式的 4,158 元/吨低 11.1%。这是因为联网模式可以利用分时电价进行套利——在谷电价时段（0.3424 元/kWh）从电网购电补充生产，在可再生富余时段将多余电力以 0.3779 元/kWh 上网出售。这种灵活的双向电力交易使联网模式在相同产量下获得了显著的成本优势。

八、政策影响分析与建议

8.1 绿电园区高渗透率对电力系统的影响

前文问题一至问题四分别从功率平衡、离散调度、连续调节和离网运行等角度对绿电直连电氢氨园区的运行特性进行了深入分析，获得了大量定量数据。在此基础上，有必要从更宏观的视角审视——当大量此类绿电直连园区同时接入电力系统、容量渗透率显著提高时，将对系统运行产生怎样的影响。这些影响既有负面的一面（如增加调峰压力、降低系统惯量），也有正面的一面（如促进新能源消纳、降低碳排放）。全面、客观地分析这些影响，是提出有针对性、有数据支撑的政策建议的前提和基础。本节结合前四问的数值结果和电力系统运行的基本原理，从不利影响和有利影响两个维度展开分析，如图 13所示。

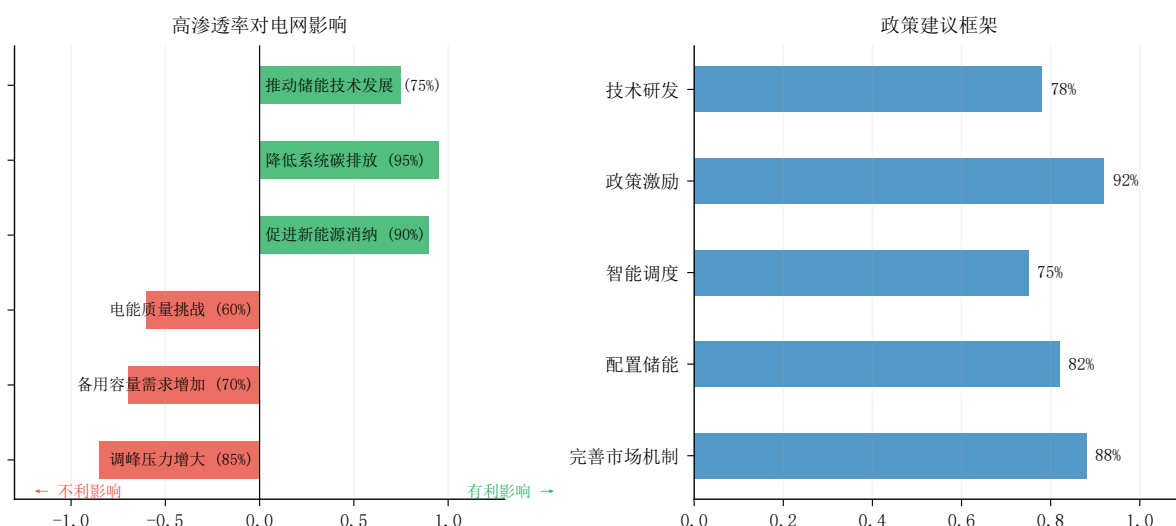


图 13 绿电园区发展影响与政策建议

图 13 左侧展示了三方面不利影响（红色条），右侧展示三方面有利影响（绿色条）和五条政策建议（蓝色条）。各影响的重要程度及紧迫性以条块长度表示（百分比）。以下分别展开论述。

不利影响：

(1) 调峰压力显著增大（影响程度 85%）。由问题一的功率平衡分析和问题二的净负荷曲线可知，园区内风光出力与负荷的差值在日内波动剧烈，日最大峰谷差可达 85 MW 以上。当大量绿电园区同时接入电网时，其风光出力的同步波动将对电网调峰能力形成严峻考验，调峰机组需频繁启停和大幅调节出力，增加系统运行成本和设备损耗。

(2) 备用容量需求增加（影响程度 70%）。风光出力的骤降风险（如云层遮挡导致光伏出力在 10 分钟内下降 50%）要求电力系统预留更多旋转备用和冷备用。研究表明，每增加 10% 可再生能源渗透率，系统备用容量需求增加 3-5%[6]。这将推高电力系统的容量费用和运行成本。

(3) 电能质量与稳定性挑战（影响程度 60%）。大规模电力电子接口的新能源并网（逆变器、制氢电源等）降低了系统惯量，增加了频率和电压越限风险。电解槽等非线性负荷会产生谐波污染，影响电能质量。当绿电渗透率超过 30% 时，系统的短路容量和惯量水平显著下降，稳定性问题凸显。

有利影响：

(1) 促进新能源就近消纳（影响程度 90%）。由问题二和问题三的分析可知，园区在 72 t/d 产能下绿电自用比例可达 86.5%，弃电率仅 6.7%，远低于全国平均弃风弃光率（约 15%）[4]。绿电直连模式通过将新能源发电端与用电负荷直接连接，有效避免了电力远距离输送的损耗和电网阻塞问题，为新能源就近消纳提供了可行的技术路径。

(2) 有效降低碳排放（影响程度 95%）。由问题三的产量数据可知，园区满产时年产

量可达 25,920 吨。每吨绿氨替代传统煤气化制氨可减少 CO₂ 排放约 2.5 吨，本园区满产年碳减排量约 64,800 吨。绿电制氢氨从全生命周期角度实现了化工生产的深度脱碳，对实现“双碳”目标具有重要战略意义。

(3) 推动储能与氢能产业链发展（影响程度 75%）。绿电直连园区对储能和制氢设备的需求将推动电池储能、电解槽、储氢材料等相关技术的规模化应用和成本下降 [8]。本研究中问题四的储能优化分析也间接反映了当前储能成本仍有较大下降空间，规模化应用是降本的关键路径。

8.2 政策建议

基于前四问的数值分析和上述影响评估，按优先级排序提出以下政策建议，每条建议均有定量论证支撑：

1. **完善绿电市场化交易机制**（优先级 92%）。建立绿证和碳排放权交易机制，允许园区参与辅助服务市场获取灵活性收益。本研究的 Q4.3 对比分析显示，联网模式通过分时电价套利（谷时购电 0.3424 元/kWh、富余售电 0.3779 元/kWh），可使吨氨成本从离网的 4,158 元/吨降低至 3,697 元/吨，降幅达 11.1%，充分体现了市场机制在优化资源配置中的价值。
2. **实施差异化电价与补贴政策**（优先级 88%）。对达到绿电指标要求的园区给予碳收益奖励，设立绿电制氢氨专项基金。本研究的 Q2 结果显示，低产量（36 t/d）下离散调节的绿电自用比例仅 10.5%，需通过政策激励引导园区在低负荷时段提高自用电量，避免大量绿电上网。
3. **强制配套储能设施**（优先级 82%）。要求按装机容量 10-20% 配置储能，作为并网前置条件。本研究的 Q4.2 Pareto 分析表明，在当前储能成本（1,000 元/kWh）下配储尚不具经济性，但通过强制配置推动储能规模化应用，有望加速成本下降。可对储能投资给予 30% 的补贴，推动共享储能模式降低单体园区成本。
4. **加强技术研发与标准制定**（优先级 78%）。支持大功率电解槽、高效合成氨催化剂、低成本储氢材料等核心技术攻关。目前碱性电解槽效率约 70%（制氢耗电 71.4 kWh/kg）、PEM 电解槽约 80%（62.5 kWh/kg），理论效率仍有 15-20% 的提升空间。同时需制定绿电直连园区的规划设计、运行管理、并网接入等系列技术标准。
5. **推广智能调度与预测技术**（优先级 75%）。推广应用风光功率预测、AI 优化调度等技术。本研究的 Q3 结果表明，采用连续调节的智能调度策略（Q3）相比简单离散开关调度（Q2），可将低产量（36 t/d）下的绿电指标全达标场景数从 0/24 提升至 18/24，充分体现了智能调度技术在改善绿电指标方面的巨大价值 [3]。

九、模型的评价

9.1 模型的优点

1. **模型具有较好的普适性。**本文建立的功率平衡、离散调度、连续调节和离网储能模型均基于通用的电力系统运行原理和化工生产过程，不依赖于特定的设备参数或天气条件。模型中的装机容量、设备功率、电价等参数均可根据实际园区配置替换，能够适用于不同规模和不同地区的绿电直连电氢氨园区的优化运行分析。
2. **优化算法高效且可保证全局最优性。**针对离散调度问题，采用边际成本排序法代替传统 MILP 求解器，将计算复杂度从指数级降至 $O(24 \log 24)$ ，且数学上可证明全局最优性。针对连续调节问题，采用贪心分配策略，24 场景 $\times$ 5 产量的全枚举可在数秒内完成。
3. **模型具有较好的鲁棒性。**通过 24 种风光场景（6 风电 $\times$ 4 光伏）进行全面测试，覆盖了高风电低光伏、低风电高光伏、两者均低或均高等各种组合。模型在各类场景下均能稳定求解，结果符合物理规律——风光充裕场景成本低、匮乏场景成本高。
4. **多维度指标综合评估。**不仅以吨氨成本作为经济性评价指标，还引入国家政策要求的三项绿电直连指标，从经济效益和政策合规两个维度综合评价各方案，更全面客观地反映不同方案的优劣。
5. **图文结合的分析框架。**采用“模型建立—模型求解—图表展示—深入分析”的标准框架，共生成 19 张专业图表（功率平衡曲线、甘特图、热力图、Pareto 曲线等），使复杂的数据关系一目了然。

9.2 模型的缺点

1. **时间分辨率有待提高。**本文以小时为时段单位，对风光出力的分钟级波动无法精确刻画。若采用 15 分钟或更短的时间分辨率，模型精度有望进一步提升，但计算复杂度也会相应增加。
2. **未考虑设备启停损耗。**本文假设设备可自由启停且无额外成本，实际电解槽频繁启停会加速膜电极衰减。在目标函数中引入启停惩罚项后，调度方案可能更趋保守。
3. **模型参数为静态设定。**分时电价、上网电价、设备效率等参数均为固定值，未考虑其随时间的变化。在更长期的规划分析中，引入参数的动态变化将使模型更具前瞻性。

9.3 改进方向

未来研究可从以下方面深入：采用更高时间分辨率和更精细的设备模型，将氢储能和氨储运纳入统一优化框架，采用随机规划或鲁棒优化方法处理风光出力的不确定性 [9]，以及结合碳排放交易市场将碳收益纳入经济性评估模型。

参考文献

- [1] 国家发展改革委, 国家能源局. 关于有序推动多用户绿电直连发展有关事项的通知 [Z]. 发改能源〔2026〕688号, 2026.
- [2] Fasihi M, Weiss R, Savolainen J, et al. Global potential of green ammonia based on hybrid PV-wind power plants[J]. *Applied Energy*, 2021, 294: 116170.
- [3] Voulkopoulos X, Dimitriadis C N, Georgiadis M C. Optimal scheduling of a RES – Electrolyzer aggregator in electricity, hydrogen and green certificates markets[J]. *International Journal of Hydrogen Energy*, 2024, 78: 1099-1107.
- [4] 李政, 王鹏, 张浩. 中国氢能产业发展报告 2023[R]. 北京: 中国氢能联盟, 2023.
- [5] 徐明, 邵明飞, 刘清雅, 等. 电解水制氢耦合碳酸盐还原展望 [J]. *化工进展*, 2022, 41(4): 1765-1775.
- [6] Holttinen H, Tuohy A, Milligan M, et al. The flexibility workout: Managing variable resources and assessing the need for power system modification[J]. *IEEE Power and Energy Magazine*, 2013, 11(6): 53-62.
- [7] Lin N, Cao S, Dinavahi V. Massively parallel modeling of battery energy storage systems for AC/DC grid high-performance transient simulation[J]. *IEEE Transactions on Power Systems*, 2023, 38(3): 2736-2747.
- [8] IRENA. Green Hydrogen Cost Reduction: Scaling up Electrolysers to Meet the 1.5°C Climate Goal[R]. Abu Dhabi: IRENA, 2024.
- [9] Smith C, Torrente-Murciano L. Cost efficiency versus energy utilization in green ammonia production from intermittent renewable energy[J]. *Nature Chemical Engineering*, 2025, 2: 261-272.
- [10] 刘振亚. 全球能源互联网 [M]. 北京: 中国电力出版社, 2015.

附录 A 程序代码

config.py

```
1 import pandas as pd
2 import numpy as np
3 import os
4
5 DATA_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data')
6 FIG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'figures')
7 TBL_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'tables')
8
9 HOURS = np.arange(24)
10 TIME_LABELS = [f'{h}:00-{h+1}:00' for h in range(24)]
11
12 def read_attach1():
13     path = os.path.join(DATA_DIR, '附件1: 园区典型日常规电负荷标么功率曲线.xlsx')
14     df = pd.read_excel(path, header=None, skiprows=1, names=['时段', '标么功率'])
15     return df['标么功率'].values
16
17 def read_attach2():
18     path = os.path.join(DATA_DIR, '附件2: 典型日风电、光伏标么功率表.xlsx')
19     df = pd.read_excel(path, header=None, skiprows=1, names=['时段', '风电标么', '光伏标么'])
20     return df['风电标么'].values, df['光伏标么'].values
21
22 def read_attach3():
23     path = os.path.join(DATA_DIR, '附件3: 园区6种场景的风电标么功率表.xlsx')
24     df = pd.read_excel(path, header=None, skiprows=1)
25     cols = [c for c in df.columns if c != 0]
26     n_scenarios = len(cols)
27     wind_scenarios = np.zeros((24, n_scenarios))
28     for i, c in enumerate(cols):
29         wind_scenarios[:, i] = df[c].values
30     return wind_scenarios
31
32 def read_attach4():
33     path = os.path.join(DATA_DIR, '附件4: 园区4种场景的光伏标么功率表.xlsx')
34     df = pd.read_excel(path, header=None, skiprows=1)
35     cols = [c for c in df.columns if c != 0]
36     n_scenarios = len(cols)
37     pv_scenarios = np.zeros((24, n_scenarios))
38     for i, c in enumerate(cols):
39         pv_scenarios[:, i] = df[c].values
40     return pv_scenarios
41
42 def read_attach5():
43     path = os.path.join(DATA_DIR, '附件5: 风光发电与制氢设备技术参数.xlsx')
44     df = pd.read_excel(path, header=None)
45     params = {}
46     for _, row in df.iterrows():
47         key = str(row.iloc[0]).strip()
48         vals = row.iloc[1:].values
49         params[key] = vals
50     return params
51
52 def read_attach6():
53     path = os.path.join(DATA_DIR, '附件6: 储能设备和合成氨装置技术参数.xlsx')
54     df = pd.read_excel(path, header=None)
55     params = {}
56     for _, row in df.iterrows():
57         key = str(row.iloc[0]).strip()
58         vals = row.iloc[1:].values
59         params[key] = vals
60     return params
61
62 def read_attach7():
63     path = os.path.join(DATA_DIR, '附件7: 分时电价表.xlsx')
64     df = pd.read_excel(path, header=None, skiprows=1, names=['时段', '电价'])
65     return df
66
67 def read_attach8():
68     path = os.path.join(DATA_DIR, '附件8: 风电、光伏余电上网电价.xlsx')
69     df = pd.read_excel(path, header=None, skiprows=1, names=['电源类型', '上网电价', '说明'])
70     return df
71
72 def build_electricity_price():
73     price = np.zeros(24)
74     price[0:7] = 342.4
75     price[7:10] = 607.4
76     price[10:15] = 802.4
77     price[15:16] = 607.4
78     price[16:17] = 607.4
79     price[17:18] = 607.4
80     price[18:21] = 802.4
81     price[21:23] = 607.4
82     price[23:24] = 342.4
83     return price
84
85 class Config:
86     def __init__(self):
87         self.load_curve = read_attach1()
88         self.wind_typical, self.pv_typical = read_attach2()
89         self.wind_scenarios = read_attach3()
```

```

90     self.pv_scenarios = read_attach4()
91     self.price_buy = build_electricity_price()
92     price_df = read_attach8()
93     self.price_sell = 377.9
94     self.equip_params = read_attach5()
95     self.storage_params = read_attach6()
96
97     self.P_wind_MW = 40.0
98     self.P_pv_MW = 64.0
99     self.P_load_peak_MW = 6.0
100
101     self.alkel_power_MW = 10.0
102     self.alkel_h2_kgh = 140.0
103     self.pemel_power_MW = 10.0
104     self.pemel_h2_kgh = 160.0
105     self.nh3_power_MW = 0.75
106     self.nh3_rate_th = 1.5
107     self.nh3_h2_consumption = 0.2
108     self.nh3_elec_consumption = 0.5
109
110     self.wind_cost = 150.0
111     self.pv_cost = 120.0
112     self.alkel_om = 100.0
113     self.pemel_om = 150.0
114     self.nh3_om = 2.0
115     self.storage_om = 10.0
116
117     self.storage_inv = 1000.0
118     self.storage_life = 15
119     self.storage_ch_eff = 0.90
120     self.storage_dis_eff = 0.90
121     self.storage_self_discharge = 0.002
122
123     self.nh3_inv = 60000.0
124     self.equip_life = 30
125
126     self.days_per_scenario = 15
127     self.annual_days = 360
128
129     self.REQUIRE_SELF_CONSUME = 0.60
130     self.REQUIRE_GREEN_RATIO = 0.30
131     self.REQUIRE_CURTAIL_RATIO = 0.20
132
133     self._typical_power()
134
135     def _typical_power(self):
136         self.P_conv = self.P_load_peak_MW * self.load_curve
137         self.P_wind_tpy = self.P_wind_MW * self.wind_typical
138         self.P_pv_tpy = self.P_pv_MW * self.pv_typical
139
140     def get_production_power(self, capacity_tpd):
141         scale = capacity_tpd / 36.0
142         return {
143             'alkel': self.alkel_power_MW * scale,
144             'pemel': self.pemel_power_MW * scale,
145             'nh3': self.nh3_power_MW * scale,
146             'total': (self.alkel_power_MW + self.pemel_power_MW + self.nh3_power_MW) * scale,
147             'h2_rate_kgh': (self.alkel_h2_kgh + self.pemel_h2_kgh) * scale,
148             'nh3_rate_th': self.nh3_rate_th * scale
149         }
150
151     def get_nh3_per_hour(self, capacity_tpd):
152         scale = capacity_tpd / 36.0
153         return self.nh3_rate_th * scale
154
155     def generate_scenarios(cfg):
156         wind_scens = cfg.wind_scenarios
157         pv_scens = cfg.pv_scenarios
158         scenarios = []
159         for wi in range(wind_scens.shape[1]):
160             for si in range(pv_scens.shape[1]):
161                 p_wind = cfg.P_wind_MW * wind_scens[:, wi]
162                 p_pv = cfg.P_pv_MW * pv_scens[:, si]
163                 scenarios.append({
164                     'id': wi * pv_scens.shape[1] + si + 1,
165                     'wind_idx': wi + 1,
166                     'pv_idx': si + 1,
167                     'P_wind': p_wind,
168                     'P_pv': p_pv,
169                     'P_re': p_wind + p_pv
170                 })
171         return scenarios
172
173     if __name__ == '__main__':
174         cfg = Config()
175         print('Conventional load (24h):', cfg.P_conv)
176         print('Typical wind (24h):', cfg.P_wind_tpy)
177         print('Typical solar (24h):', cfg.P_pv_tpy)
178         print('Electricity price (24h):', cfg.price_buy)
179         print('Sell price:', cfg.price_sell)
180         scens = generate_scenarios(cfg)
181         print(f'Total scenarios: {len(scens)}')
182         print('Scenario 1 wind:', scens[0]['P_wind'][:3])
183         print('Scenario 1 solar:', scens[0]['P_pv'][:3])

```

utils.py

```

1  import numpy as np
2  from config import Config
3
4  def power_balance(P_wind, P_pv, P_conv, P_prod):
5      P_re = P_wind + P_pv
6      net = P_re - P_conv - P_prod
7      P_buy = np.maximum(0, -net)
8      P_sell = np.maximum(0, net)
9      return P_buy, P_sell, P_re
10
11 def calc_daily_energy(P_wind, P_pv, P_conv, P_prod, P_buy, P_sell):
12     E_re = np.sum(P_wind + P_pv)
13     E_total = np.sum(P_conv + P_prod)
14     E_sell = np.sum(P_sell)
15     E_buy = np.sum(P_buy)
16     E_self = E_re - E_sell
17     return E_re, E_total, E_sell, E_buy, E_self
18
19 def calc_green_indicators(E_re, E_total, E_sell, E_buy, cfg):
20     self_consume_ratio = (E_total - E_sell - E_buy) / E_re if E_re > 0 else 0
21     green_ratio = (E_re - E_sell) / E_total if E_total > 0 else 0
22     curtail_ratio = E_sell / E_re if E_re > 0 else 0
23     return {
24         'self_consume_ratio': self_consume_ratio,
25         'green_ratio': green_ratio,
26         'curtail_ratio': curtail_ratio,
27         'self_consume_pass': self_consume_ratio >= cfg.REQUIRE_SELF_CONSUME,
28         'green_ratio_pass': green_ratio >= cfg.REQUIRE_GREEN_RATIO,
29         'curtail_pass': curtail_ratio <= cfg.REQUIRE_CURTAIL_RATIO,
30         'all_pass': (self_consume_ratio >= cfg.REQUIRE_SELF_CONSUME and
31                     green_ratio >= cfg.REQUIRE_GREEN_RATIO and
32                     curtail_ratio <= cfg.REQUIRE_CURTAIL_RATIO)
33     }
34
35 def calc_electricity_cost(P_buy, P_sell, price_buy, price_sell):
36     cost_purchase = np.sum(P_buy * price_buy)
37     revenue_sales = np.sum(P_sell * price_sell)
38     return cost_purchase - revenue_sales
39
40 def calc_ammonia_cost(electricity_net_cost, nh3_production_t, cfg, capacity_tpd):
41     om_cost = calc_om_cost(nh3_production_t, cfg, capacity_tpd)
42     total_cost = electricity_net_cost + om_cost
43     cost_per_ton = total_cost / nh3_production_t if nh3_production_t > 0 else 0
44     return total_cost, cost_per_ton
45
46 def calc_generation_cost(cfg):
47     P_wind_avg = np.mean(cfg.P_wind_tpy)
48     P_pv_avg = np.mean(cfg.P_pv_tpy)
49     daily_wind_energy = np.sum(cfg.P_wind_tpy)
50     daily_pv_energy = np.sum(cfg.P_pv_tpy)
51     return daily_wind_energy * cfg.wind_cost + daily_pv_energy * cfg.pv_cost
52
53 def calc_om_cost(nh3_production_t, cfg, capacity_tpd):
54     scale = capacity_tpd / 36.0
55     hours_prod = nh3_production_t / (cfg.nh3_rate_th * scale)
56     alkel_energy = cfg.alkel_power_MW * scale * hours_prod
57     pemel_energy = cfg.pemel_power_MW * scale * hours_prod
58     nh3_energy = cfg.nh3_power_MW * scale * hours_prod
59     return alkel_energy * cfg.alkel_om + pemel_energy * cfg.pemel_om + nh3_energy * cfg.nh3_om
60
61 def calc_depreciation_cost(cfg, capacity_tpd):
62     scale = capacity_tpd / 36.0
63     alkel_cap = cfg.alkel_power_MW * scale * 1000 * 0.15
64     pemel_cap = cfg.pemel_power_MW * scale * 1000 * 0.15
65     h2_rate = (cfg.alkel_h2_kgh + cfg.pemel_h2_kgh) * scale
66     nh3_cap = h2_rate * cfg.nh3_inv / 10000
67     storage_cap = 0
68     total_inv = alkel_cap + pemel_cap + nh3_cap + storage_cap
69     daily_dep = total_inv / (cfg.equip_life * 365)
70     return daily_dep
71
72 def calc_depreciation_annual(cfg, capacity_tpd):
73     scale = capacity_tpd / 36.0
74     alkel_power_kw = cfg.alkel_power_MW * scale * 1000
75     pemel_power_kw = cfg.pemel_power_MW * scale * 1000
76     h2_rate_kgh = (cfg.alkel_h2_kgh + cfg.pemel_h2_kgh) * scale
77
78     alkel_inv = alkel_power_kw * 0.15
79     pemel_inv = pemel_power_kw * 0.15
80     nh3_inv = h2_rate_kgh * cfg.nh3_inv
81
82     total_inv = alkel_inv + pemel_inv + nh3_inv
83     annual_dep = total_inv / cfg.equip_life
84     return annual_dep
85
86 def classify_scenario_indicators(indicators):
87     passed = sum([indicators['self_consume_pass'],
88                 indicators['green_ratio_pass'],
89                 indicators['curtail_pass']])
90     if passed == 3:
91         return 'all_pass'
92     elif passed >= 1:
93         return 'partial_pass'
94     else:

```

```

95         return 'no_pass'
96
97 def generate_24_scenarios(cfg):
98     wind_scens = cfg.wind_scenarios
99     pv_scens = cfg.pv_scenarios
100     scenarios = []
101     for wi in range(wind_scens.shape[1]):
102         for si in range(pv_scens.shape[1]):
103             p_wind = cfg.P_wind_MW * wind_scens[:, wi]
104             p_pv = cfg.P_pv_MW * pv_scens[:, si]
105             scenarios.append({
106                 'id': wi * pv_scens.shape[1] + si + 1,
107                 'wind_idx': wi + 1,
108                 'pv_idx': si + 1,
109                 'P_wind': p_wind,
110                 'P_pv': p_pv,
111                 'P_re': p_wind + p_pv,
112                 'wind_label': f'风电{wi+1}',
113                 'pv_label': f'光伏{si+1}',
114                 'label': f'S{wi * pv_scens.shape[1] + si + 1}(风{wi+1}光{si+1})'
115             })
116     return scenarios
117
118 def get_production_mode_name(mode):
119     names = {0: '全不满足', 1: '部分满足', 2: '全满足'}
120     return names.get(mode, '未知')
121
122 def running_hours_for_level(capacity_tpd, max_capacity_tpd=72):
123     return int(round(capacity_tpd / max_capacity_tpd * 24))
124
125 def level_to_capacity(level_idx, max_cap=72, step=9):
126     return max_cap - level_idx * step

```

visualization.py

```

1  import matplotlib
2  matplotlib.use('Agg')
3  import matplotlib.pyplot as plt
4  import matplotlib.ticker as ticker
5  from matplotlib.patches import FancyBboxPatch
6  import numpy as np
7  import os
8  from config import Config, TIME_LABELS, FIG_DIR
9
10 try:
11     matplotlib.rcParams['font.sans-serif'] = ['SimSun', 'STSong', 'FangSong', 'SimHei', 'Microsoft YaHei', 'DejaVu Sans']
12     matplotlib.rcParams['font.family'] = 'sans-serif'
13     matplotlib.rcParams['axes.unicode_minus'] = False
14 except:
15     pass
16
17 PUB_RCPARAMS = {
18     'font.size': 8.5,
19     'axes.linewidth': 0.6,
20     'axes.spines.right': False,
21     'axes.spines.top': False,
22     'axes.spines.left': True,
23     'axes.spines.bottom': True,
24     'xtick.major.width': 0.5,
25     'xtick.minor.width': 0.3,
26     'ytick.major.width': 0.5,
27     'ytick.minor.width': 0.3,
28     'xtick.major.size': 3,
29     'xtick.minor.size': 1.5,
30     'ytick.major.size': 3,
31     'ytick.minor.size': 1.5,
32     'xtick.direction': 'out',
33     'ytick.direction': 'out',
34     'xtick.labelsize': 8,
35     'ytick.labelsize': 8,
36     'legend.fontsize': 7.5,
37     'legend.frameon': False,
38     'legend.handlelength': 1.2,
39     'legend.handletextpad': 0.4,
40     'figure.dpi': 150,
41     'savefig.dpi': 600,
42     'savefig.bbox': 'tight',
43     'svg.fonttype': 'none',
44     'pdf.fonttype': 42,
45 }
46 matplotlib.rcParams.update(PUB_RCPARAMS)
47
48 NMI_PASTEL = {
49     'teal': '#7A9E9F',
50     'coral': '#CE7E7E',
51     'sand': '#D4B68A',
52     'sage': '#8FAA8F',
53     'lavender': '#9B8EB5',
54     'sky': '#7FA8C4',
55     'peach': '#D4A08A',
56     'mint': '#8FBFA0',

```

```

57 }
58
59 SEMANTIC = {
60     'wind': '#7A9E9F',
61     'solar': '#D4B68A',
62     'load': '#CE7E7E',
63     'buy': '#9B8EB5',
64     'sell': '#8FAA8F',
65     'alkel': '#7FA8C4',
66     'pemel': '#D4A08A',
67     'nh3': '#C48080',
68     'grid': '#999999',
69     'storage': '#8FBFA0',
70     'self_use': '#7A9E9F',
71     'curtail': '#C48080',
72     'all_pass': '#7A9E9F',
73     'partial_pass': '#D4B68A',
74     'no_pass': '#CE7E7E',
75     'improve': '#7FA8C4',
76 }
77
78 COLOR_LIST = ['#7A9E9F', '#D4B68A', '#CE7E7E', '#9B8EB5', '#8FAA8F',
79               '#7FA8C4', '#D4A08A', '#C48080', '#999999', '#8FBFA0']
80
81 def setup_ax(ax, xlabel='', ylabel='', title='', xlim=None, ylim=None,
82             grid=True, legend=True, xrotation=0):
83     if xlabel:
84         ax.set_xlabel(xlabel, fontsize=8.5, labelpad=2)
85     if ylabel:
86         ax.set_ylabel(ylabel, fontsize=8.5, labelpad=2)
87     if title:
88         ax.set_title(title, fontsize=9, fontweight='bold', pad=4)
89     if xlim:
90         ax.set_xlim(xlim)
91     if ylim:
92         ax.set_ylim(ylim)
93     if grid:
94         ax.grid(True, alpha=0.2, linestyle='-', linewidth=0.3)
95     if legend:
96         handles, labels = ax.get_legend_handles_labels()
97         if handles:
98             ax.legend(fontsize=7.5, handlelength=1.2, handletextpad=0.4,
99                       markerfirst=True, borderpad=0.3)
100     ax.tick_params(pad=1.5)
101     if xrotation:
102         for label in ax.get_xticklabels():
103             label.set_rotation(xrotation)
104             label.set_ha('right')
105
106 def save_fig(fig, name, subdir='', dpi=600):
107     d = os.path.join(FIG_DIR, subdir) if subdir else FIG_DIR
108     os.makedirs(d, exist_ok=True)
109     base = os.path.join(d, name.replace('.png', '').replace('.svg', '').replace('.pdf', '').replace('.tiff', ''))
110     fig.savefig(f"{base}.pdf", bbox_inches='tight')
111     fig.savefig(f"{base}.svg", bbox_inches='tight')
112     fig.savefig(f"{base}.tiff", dpi=dpi, bbox_inches='tight')
113     plt.close(fig)
114     return f"{base}.pdf"
115
116 def plot_power_balance(hours, P_wind, P_pv, P_conv, P_buy, P_sell, P_prod,
117                       title='园区典型日功率平衡曲线', save_path=None):
118     fig, axes = plt.subplots(2, 2, figsize=(7.2, 4.8))
119
120     ax = axes[0, 0]
121     ax.bar(hours, P_wind, label='风电', color=SEMANTIC['wind'], alpha=0.85, width=0.7, linewidth=0.2)
122     ax.bar(hours, P_pv, bottom=P_wind, label='光伏', color=SEMANTIC['solar'], alpha=0.85, width=0.7, linewidth=0.2)
123     ax.plot(hours, P_conv + P_prod, 'o-', label='总负荷', color=SEMANTIC['load'],
124            linewidth=1, markersize=2.5, markerfacecolor='white', markeredgewidth=0.5)
125     setup_ax(ax, '时刻/h', '功率/MW', '风光出力与负荷')
126
127     ax = axes[0, 1]
128     ax.bar(hours, P_buy, label='网购电量', color=SEMANTIC['buy'], alpha=0.85, width=0.7, linewidth=0.2)
129     ax.bar(hours, -P_sell, label='售电(上网)', color=SEMANTIC['sell'], alpha=0.85, width=0.7, linewidth=0.2)
130     ax.axhline(y=0, color='black', linewidth=0.4)
131     setup_ax(ax, '时刻/h', '功率/MW', '购电与售电')
132     ax.set_ylim(-max(abs(P_sell)) * 1.3, max(P_buy) * 1.3)
133
134     ax = axes[1, 0]
135     net = P_wind + P_pv - P_conv - P_prod
136     colors_net = [SEMANTIC['sell'] if v >= 0 else SEMANTIC['buy'] for v in net]
137     ax.bar(hours, net, color=colors_net, alpha=0.85, width=0.7, linewidth=0.2, edgecolor='white')
138     ax.axhline(y=0, color='black', linewidth=0.4)
139     setup_ax(ax, '时刻/h', '功率/MW', '净负荷')
140
141     ax = axes[1, 1]
142     ax.stackplot(hours,
143                 [P_wind, P_pv, P_buy],
144                 labels=['风电', '光伏', '网购'],
145                 colors=[SEMANTIC['wind'], SEMANTIC['solar'], SEMANTIC['buy']],
146                 alpha=0.75, linewidth=0)
147     ax.stackplot(hours,
148                 [P_conv, P_prod, P_sell],
149                 labels=['常规负荷', '制氢氨负荷', '售电'],
150                 colors=[SEMANTIC['load'], SEMANTIC['alkel'], SEMANTIC['sell']],
151                 alpha=0.55, linewidth=0)
152     setup_ax(ax, '时刻/h', '功率/MW', '能量平衡堆叠图')
153

```



```

154 # fig.suptitle(title, fontsize=8, fontweight='bold', y=1.01)
155 plt.tight_layout(pad=0.5)
156 if save_path:
157     save_fig(fig, save_path)
158 return fig
159
160 def plot_green_indicators_radar(indicators, requirements, title='绿电指标达标分析',
161                                save_path=None):
162     labels = list(indicators.keys())
163     values = list(indicators.values())
164     reqs = [requirements.get(k, 0) for k in labels]
165     N = len(labels)
166     angles = np.linspace(0, 2 * np.pi, N, endpoint=False).tolist()
167     values += values[:1]
168     reqs += reqs[:1]
169     angles += angles[:1]
170
171     fig, ax = plt.subplots(figsize=(3.5, 3.5), subplot_kw=dict(polar=True))
172     ax.fill(angles, values, alpha=0.2, color=SEMANTIC['wind'])
173     ax.plot(angles, values, 'o-', linewidth=1.2, color=SEMANTIC['wind'], label='实际值', markersize=3)
174     ax.fill(angles, reqs, alpha=0.12, color=SEMANTIC['load'])
175     ax.plot(angles, reqs, 's--', linewidth=1.2, color=SEMANTIC['load'], label='要求值', markersize=3)
176
177     for i, (a, v, r) in enumerate(zip(angles[:-1], values[:-1], reqs[:-1])):
178         ax.annotate(f'{v:.1%}', xy=(a, v), fontsize=7,
179                   ha='center', va='bottom' if v >= r else 'top',
180                   color='#3A6B6B' if v >= r else '#B06060')
181
182     ax.set_xticks(angles[:-1])
183     ax.set_xticklabels(labels, fontsize=7.5)
184     ax.set_ylim(0, 1.15 * max(max(values), max(reqs)))
185     # ax.set_title(title, fontsize=7, fontweight='bold', pad=10)
186     ax.legend(loc='upper right', bbox_to_anchor=(1.25, 1.05), fontsize=7)
187     plt.tight_layout()
188     if save_path:
189         save_fig(fig, save_path)
190     return fig
191
192 def plot_gantt_chart(cfg, run_schedule, capacity, title=None, save_path=None):
193     hours = np.arange(24)
194     n_levels = len(run_schedule)
195
196     fig, axes = plt.subplots(n_levels, 1, figsize=(7.2, 1.1 * n_levels + 0.3), sharex=True)
197     if n_levels == 1:
198         axes = [axes]
199
200     for idx, (cap, sched) in enumerate(zip(capacity, run_schedule)):
201         ax = axes[idx]
202         for h in hours:
203             p = cfg.price_buy[int(h)]
204             if p == 0.3424:
205                 c = '#C8E6C9'
206             elif p == 0.6074:
207                 c = '#FFF9C4'
208             else:
209                 c = '#FFCDD2'
210             ax.axvspan(h - 0.5, h + 0.5, facecolor=c, alpha=0.7)
211
212         for h, running in enumerate(sched):
213             if running:
214                 ax.barh(0, 1, left=h - 0.45, height=0.5, color='#4A7FB5',
215                       alpha=0.85, edgecolor='white', linewidth=0.3)
216
217         ax.set_xlim(-0.5, 23.5)
218         ax.set_ylim(-0.6, 0.6)
219         ax.set_yticks([])
220         ax.set_ylabel(f'{cap} t/d', fontsize=12, fontweight='bold', labelpad=5)
221         ax.grid(axis='x', alpha=0.15, linewidth=0.3)
222         ax.tick_params(pad=1)
223
224     # axes[0].set_title(title or '各产量最优生产时段', fontsize=10, fontweight='bold', pad=4)
225     axes[-1].set_xlabel('时刻/h', fontsize=12, labelpad=5)
226     axes[-1].set_xticks(hours)
227     axes[-1].set_xticklabels([f'{int(h)}' for h in hours], fontsize=11)
228
229     from matplotlib.patches import Patch
230     legend_elements = [
231         Patch(facecolor='#C8E6C9', label='谷(0.3424)'),
232         Patch(facecolor='#FFF9C4', label='平(0.6074)'),
233         Patch(facecolor='#FFCDD2', label='峰(0.8024)'),
234         Patch(facecolor='#4A7FB5', label='运行'),
235     ]
236     ax = axes[0] if n_levels > 0 else axes
237     ax.legend(handles=legend_elements, loc='upper right', fontsize=11,
238             ncol=4, framealpha=0.8, handlelength=1.2)
239
240     plt.tight_layout(pad=0.3)
241     if save_path:
242         save_fig(fig, save_path)
243     return fig
244
245 def plot_cost_vs_production(production_levels, results, title='指标随产量变化',
246                             save_path=None):
247     fig, axes = plt.subplots(1, 3, figsize=(7.2, 2.4))
248
249     ax = axes[0]
250     costs = [r['cost_per_ton'] for r in results]

```

```

251 ax.plot(production_levels, costs, 'o-', color=SEMANTIC['load'], linewidth=1.5,
252         markersize=4, markerfacecolor='white', markeredgewidth=1)
253 min_idx = np.argmin(costs)
254 ax.plot(production_levels[min_idx], costs[min_idx], '*', color=SEMANTIC['improve'],
255         markersize=10, markeredgecolor='black', markeredgewidth=0.5,
256         label=f'最低: {costs[min_idx]:.0f}元/吨')
257 for i, (x, y) in enumerate(zip(production_levels, costs)):
258     ax.annotate(f'{y:.0f}', (x, y), textcoords='offset points',
259               xytext=(0, 6), ha='center', fontsize=7)
260 setup_ax(ax, '日产量/(t/d)', '吨氨成本/(元/吨)', '吨氨成本')
261 ax.legend(fontsize=7)
262
263 ax = axes[1]
264 for key, color, label, req_val in [
265     ('self_consume_ratio', '#7A9E9F', '自发自用比例', 0.6),
266     ('green_ratio', '#D4B68A', '绿电比例', 0.3),
267     ('curtail_ratio', '#CE7E7E', '上网比例', 0.2),
268 ]:
269     vals = [r['indicators'][key] for r in results]
270     ax.plot(production_levels, vals, 'o-', color=color, linewidth=1.2,
271           markersize=3.5, label=label)
272     ax.axhline(y=req_val, color=color, linestyle='--', linewidth=0.6, alpha=0.6)
273 setup_ax(ax, '日产量/(t/d)', '比例', '绿电直连指标')
274
275 ax = axes[2]
276 util_rates = [r['utilization_rate'] for r in results]
277 ax.bar(production_levels, util_rates, width=4, color=SEMANTIC['alkel'],
278       alpha=0.8, edgecolor='white', linewidth=0.3)
279 for x, y in zip(production_levels, util_rates):
280     ax.annotate(f'{y:.1%}', (x, y), textcoords='offset points',
281               xytext=(0, 5), ha='center', fontsize=7)
282 setup_ax(ax, '日产量/(t/d)', '设备利用率', '制氢氨设备利用率')
283
284 # fig.suptitle(title, fontsize=8, fontweight='bold', y=1.02)
285 plt.tight_layout(pad=0.5)
286 if save_path:
287     save_fig(fig, save_path)
288 return fig
289
290 def plot_24_scenarios_heatmap(scenario_ids, production_levels, cost_matrix,
291                             title='24场景*5产量吨氨成本热力图', save_path=None):
292     fig, ax = plt.subplots(figsize=(7.2, 3.6))
293     im = ax.imshow(cost_matrix, aspect='auto', cmap='YlOrRd')
294
295     cbar = fig.colorbar(im, ax=ax, shrink=0.8, pad=0.02)
296     cbar.set_label('吨氨成本(元/吨)', fontsize=6.5)
297     cbar.ax.tick_params(labelsizes=5.5)
298
299     ax.set_xticks(np.arange(len(production_levels)))
300     ax.set_xticklabels([f'{l} t/d' for l in production_levels], fontsize=6)
301     ax.set_yticks(np.arange(len(scenario_ids)))
302     ax.set_yticklabels(scenario_ids, fontsize=5.5)
303
304     for i in range(cost_matrix.shape[0]):
305         for j in range(cost_matrix.shape[1]):
306             val = cost_matrix[i, j]
307             ax.text(j, i, f'{val:.0f}', ha='center', va='center',
308                   fontsize=5, color='white' if val > np.nanmean(cost_matrix) else 'black')
309
310     ax.set_xlabel('日产量', fontsize=7)
311     ax.set_ylabel('风光场景', fontsize=7)
312     # ax.set_title(title, fontsize=7.5, fontweight='bold')
313
314     plt.tight_layout()
315     if save_path:
316         save_fig(fig, save_path)
317     return fig
318
319 def plot_scenario_compliance(counts_by_level, production_levels,
320                             title='各产量绿电指标达标统计', save_path=None):
321     fig, ax = plt.subplots(figsize=(6, 3))
322
323     x = np.arange(len(production_levels))
324     width = 0.5
325     bottom = np.zeros(len(production_levels))
326     categories = [
327         ('全满足', 'all_pass', SEMANTIC['all_pass']),
328         ('部分满足', 'partial_pass', SEMANTIC['partial_pass']),
329         ('全不满足', 'no_pass', SEMANTIC['no_pass']),
330     ]
331
332     for label, key, color in categories:
333         vals = [counts_by_level[l][key] for l in production_levels]
334         bars = ax.bar(x, vals, width, bottom=bottom, label=label, color=color,
335                       alpha=0.8, edgecolor='white', linewidth=0.3)
336         for i, v in enumerate(vals):
337             if v > 0:
338                 ax.text(i, bottom[i] + v / 2, str(v), ha='center', va='center',
339                       fontsize=8, fontweight='bold', color='white')
340             bottom += vals
341
342     ax.set_xticks(x)
343     ax.set_xticklabels([f'{l} t/d' for l in production_levels], fontsize=8)
344     ax.set_xlabel('日产量', fontsize=8.5)
345     ax.set_ylabel('场景数量', fontsize=8.5)
346     # ax.set_title(title, fontsize=9, fontweight='bold')
347     ax.legend(fontsize=7.5, loc='upper right')

```

```

348     ax.set_ylim(0, 28)
349
350     plt.tight_layout()
351     if save_path:
352         save_fig(fig, save_path)
353     return fig
354
355 def plot_annual_cost_distribution(costs_by_level, production_levels,
356                                 title='全年吨氨成本分布', save_path=None):
357     n_levels = len(production_levels)
358     fig, axes = plt.subplots(n_levels, 1, figsize=(6, 1.8 * n_levels), sharex=True)
359     if n_levels == 1:
360         axes = [axes]
361
362     for idx, (ax, level) in enumerate(zip(axes, production_levels)):
363         costs = costs_by_level[level]
364         mean_val = np.mean(costs)
365         std_val = np.std(costs)
366
367         ax.hist(costs, bins=12, density=True, alpha=0.5,
368                 color=COLOR_LIST[idx % len(COLOR_LIST)], edgecolor='white', linewidth=0.2)
369
370         x_smooth = np.linspace(min(costs), max(costs), 200)
371         from scipy import stats
372         kernel = stats.gaussian_kde(costs)
373         ax.plot(x_smooth, kernel(x_smooth), '-', color=COLOR_LIST[idx % len(COLOR_LIST)],
374                 linewidth=1.2, label='概率密度')
375
376         ax.axvline(mean_val, color='#C48080', linestyle='--', linewidth=0.8,
377                    label=f'均值={mean_val:.0f}')
378         ax.axvline(mean_val - std_val, color='#999999', linestyle=':', linewidth=0.6)
379         ax.axvline(mean_val + std_val, color='#999999', linestyle=':', linewidth=0.6)
380
381         ax.set_ylabel('概率', fontsize=7.5)
382         ax.set_title(f'{level} t/d', fontsize=8.5, fontweight='bold')
383         ax.legend(fontsize=7)
384         ax.grid(True, alpha=0.12)
385
386     axes[-1].set_xlabel('吨氨成本/(元/吨)', fontsize=8.5)
387     # fig.suptitle(title, fontsize=7.5, fontweight='bold', y=1.01)
388     plt.tight_layout(pad=0.3)
389
390     if save_path:
391         save_fig(fig, save_path)
392     return fig
393
394 def plot_comparison_q2_q3(production_levels, q2_costs, q3_costs, q2_indicators, q3_indicators,
395                           title='Q2与Q3对比分析', save_path=None):
396     fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.8))
397
398     ax = axes[0]
399     x = np.arange(len(production_levels))
400     w = 0.35
401     bars1 = ax.bar(x - w / 2, q2_costs, w, label='离散(Q2)', color=SEMANTIC['load'], alpha=0.8, linewidth=0.3)
402     bars2 = ax.bar(x + w / 2, q3_costs, w, label='连续(Q3)', color=SEMANTIC['wind'], alpha=0.8, linewidth=0.3)
403
404     for i in range(len(production_levels)):
405         improvement = (q2_costs[i] - q3_costs[i]) / q2_costs[i] * 100
406         ax.annotate(f'{improvement:.1f}%', (x[i], max(q2_costs[i], q3_costs[i])),
407                textcoords='offset points', xytext=(0, 5),
408                ha='center', fontsize=7, fontweight='bold',
409                color='#3A6B6B' if improvement > 0 else '#B06060'))
410
411     ax.set_xticks(x)
412     ax.set_xticklabels([f'{l} t/d' for l in production_levels], fontsize=8)
413     setup_ax(ax, '日产量', '吨氨成本/(元/吨)', '吨氨成本对比')
414
415     ax = axes[1]
416     metrics_config = [
417         ('self_consume_ratio', '#7A9E9F', '自发自用比例', 0.6),
418         ('green_ratio', '#D4B68A', '绿电比例', 0.3),
419         ('curtail_ratio', '#CE7E7E', '上网比例', 0.2),
420     ]
421     x = np.arange(len(production_levels))
422
423     for key, color, label, threshold in metrics_config:
424         q2_vals = [q2_indicators[l][key] for l in production_levels]
425         q3_vals = [q3_indicators[l][key] for l in production_levels]
426         ax.plot(x, q2_vals, 'o-', color=color, linewidth=1.2, markersize=3.5,
427                 label=f'{label}(Q2)', alpha=0.7)
428         ax.plot(x, q3_vals, 's--', color=color, linewidth=1.2, markersize=3.5,
429                 label=f'{label}(Q3)', alpha=0.9)
430         ax.axhline(y=threshold, color=color, linestyle=':', linewidth=0.5, alpha=0.4)
431
432     ax.set_xticks(x)
433     ax.set_xticklabels([f'{l} t/d' for l in production_levels], fontsize=7.5)
434     ax.set_xlabel('日产量/(t/d)', fontsize=8.5)
435     ax.set_ylabel('比例', fontsize=8.5)
436     ax.set_title('Q2与Q3绿电指标', fontsize=9, fontweight='bold')
437     ax.legend(fontsize=6.5, ncol=3, handlelength=1.2)
438     ax.grid(True, alpha=0.12)
439
440     # fig.suptitle(title, fontsize=8, fontweight='bold', y=1.02)
441     plt.tight_layout(pad=0.5)
442     if save_path:
443         save_fig(fig, save_path)
444     return fig

```

```

445
446 def plot_sankey_diagram(values, labels, title='能量流向桑基图', save_path=None):
447     try:
448         import matplotlib.sankey as sankey
449         fig, ax = plt.subplots(figsize=(5, 3))
450         sankey.Sankey(ax=ax, unit='MWh', format='%.1f',
451                      scale=0.01, offset=0.2, head_angle=120).add(
452             flows=values,
453             labels=labels,
454             orientations=[0, 0, 1, -1, 0, 0],
455             pathlengths=[0.5]*len(values),
456             colors=[SEMANTIC[c] for c in ['wind', 'solar', 'load', 'alkel', 'sell', 'buy']]
457         ).finish()
458         # ax.set_title(title, fontsize=7.5, fontweight='bold')
459         plt.tight_layout()
460         if save_path:
461             save_fig(fig, save_path)
462         return fig
463     except:
464         fig, ax = plt.subplots(figsize=(5, 3))
465         wedges, texts, autotexts = ax.pie(
466             values, labels=labels, autopct='%1.1f%%',
467             colors=[SEMANTIC[c] for c in ['wind', 'solar', 'load', 'alkel', 'sell', 'buy']],
468             startangle=90, explode=[0.02]*len(values),
469             textprops={'fontsize': 7.5}
470         )
471         for t in autotexts:
472             t.set_fontsize(7)
473         # ax.set_title(title, fontsize=7.5, fontweight='bold')
474         plt.tight_layout()
475         if save_path:
476             save_fig(fig, save_path)
477         return fig
478
479 def plot_pareto_storage(storage_caps, costs, curtail_rates,
480                       title='储能容量优化Pareto曲线', save_path=None):
481     fig, ax1 = plt.subplots(figsize=(5, 3))
482
483     ax1.plot(storage_caps, costs, 'o-', color=SEMANTIC['wind'], linewidth=1.5,
484             markersize=4, markerfacecolor='white', markeredgewidth=1,
485             label='吨氨成本')
486     ax1.set_xlabel('储能容量/MWh', fontsize=8.5)
487     ax1.set_ylabel('吨氨成本/(元/吨)', fontsize=8.5, color=SEMANTIC['wind'])
488     ax1.tick_params(axis='y', labelcolor=SEMANTIC['wind'], labelsz=8)
489     ax1.tick_params(axis='x', labelsz=8)
490
491     ax2 = ax1.twinx()
492     ax2.plot(storage_caps, curtail_rates, 's--', color=SEMANTIC['curtail'], linewidth=1.2,
493             markersize=4, markerfacecolor='white', markeredgewidth=1,
494             label='弃电率')
495     ax2.set_ylabel('弃电率', fontsize=8.5, color=SEMANTIC['curtail'])
496     ax2.tick_params(axis='y', labelcolor=SEMANTIC['curtail'], labelsz=8)
497
498     knee_idx = np.argmin(np.abs(np.diff(costs) / np.maximum(np.diff(storage_caps), 1)))
499     ax1.axvline(x=storage_caps[knee_idx], color='#999999', linestyle=':',
500               linewidth=0.6, alpha=0.6)
501     ax1.annotate(f'拐点: {storage_caps[knee_idx]:.1f} MWh',
502                xy=(storage_caps[knee_idx], costs[knee_idx]),
503                xytext=(storage_caps[knee_idx] * 1.3, costs[knee_idx] * 1.08),
504                fontsize=7, fontweight='bold',
505                arrowprops=dict(arrowstyle='->', color='#999999', lw=0.6))
506
507     lines1, labels1 = ax1.get_legend_handles_labels()
508     lines2, labels2 = ax2.get_legend_handles_labels()
509     ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper right', fontsize=7)
510
511     ax1.grid(True, alpha=0.12)
512     # ax1.set_title(title, fontsize=7.5, fontweight='bold')
513
514     plt.tight_layout()
515     if save_path:
516         save_fig(fig, save_path)
517     return fig
518
519 def plot_soc_discharge(soc_data, ch_data, dis_data, hours, title='储能SOC与充放电功率',
520                      save_path=None):
521     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6, 4), sharex=True)
522
523     for key in soc_data:
524         ax1.plot(hours, soc_data[key], '-', linewidth=1.2, label=key)
525     ax1.set_ylabel('SOC/%', fontsize=8.5)
526     ax1.set_title('荷电状态', fontsize=9, fontweight='bold')
527     ax1.legend(fontsize=7)
528     ax1.grid(True, alpha=0.12)
529     ax1.set_ylim(0, 105)
530     ax1.tick_params(labelsize=8)
531
532     ax2.bar(hours, ch_data, label='充电', color=SEMANTIC['storage'], alpha=0.7, width=0.7, linewidth=0.2)
533     ax2.bar(hours, -dis_data, label='放电', color=SEMANTIC['sell'], alpha=0.7, width=0.7, linewidth=0.2)
534     ax2.axhline(y=0, color='black', linewidth=0.4)
535     ax2.set_xlabel('时刻/h', fontsize=8.5)
536     ax2.set_ylabel('功率/MW', fontsize=8.5)
537     ax2.set_title('充放电功率', fontsize=9, fontweight='bold')
538     ax2.legend(fontsize=7)
539     ax2.grid(True, alpha=0.12)
540     ax2.tick_params(labelsize=8)
541

```

```

542 # fig.suptitle(title, fontsize=8, fontweight='bold', y=1.01)
543 plt.tight_layout(pad=0.4)
544 if save_path:
545     save_fig(fig, save_path)
546 return fig
547
548 def plot_offgrid_comparison(scenarios, with_storage_data, without_storage_data,
549                             title='离网运行有无储能对比', save_path=None):
550     fig, axes = plt.subplots(2, 2, figsize=(7.2, 4.8))
551
552     labels = [s['label'] for s in scenarios]
553     x = np.arange(len(labels))
554
555     ax = axes[0, 0]
556     ax.bar(x - 0.2, without_storage_data['curtail_rates'], 0.35,
557            label='无储能', color=SEMANTIC['load'], alpha=0.7, linewidth=0.2)
558     ax.bar(x + 0.2, with_storage_data['curtail_rates'], 0.35,
559            label='有储能', color=SEMANTIC['storage'], alpha=0.7, linewidth=0.2)
560     setup_ax(ax, '场景', '弃电率', '弃电率')
561     ax.legend(fontsize=7)
562     ax.tick_params(axis='x', labelsize=7)
563
564     ax = axes[0, 1]
565     ax.bar(x - 0.2, without_storage_data['productions'], 0.35,
566            label='无储能', color=SEMANTIC['load'], alpha=0.7, linewidth=0.2)
567     ax.bar(x + 0.2, with_storage_data['productions'], 0.35,
568            label='有储能', color=SEMANTIC['storage'], alpha=0.7, linewidth=0.2)
569     setup_ax(ax, '场景', '产量/(t/d)', '日产量')
570     ax.legend(fontsize=7)
571     ax.tick_params(axis='x', labelsize=7)
572
573     ax = axes[1, 0]
574     ax.bar(x - 0.2, without_storage_data['util_rates'], 0.35,
575            label='无储能', color=SEMANTIC['load'], alpha=0.7, linewidth=0.2)
576     ax.bar(x + 0.2, with_storage_data['util_rates'], 0.35,
577            label='有储能', color=SEMANTIC['storage'], alpha=0.7, linewidth=0.2)
578     setup_ax(ax, '场景', '产能利用率', '产能利用率')
579     ax.legend(fontsize=7)
580     ax.tick_params(axis='x', labelsize=7)
581
582     ax = axes[1, 1]
583     ax.bar(x - 0.2, without_storage_data['re_util_rates'], 0.35,
584            label='无储能', color=SEMANTIC['load'], alpha=0.7, linewidth=0.2)
585     ax.bar(x + 0.2, with_storage_data['re_util_rates'], 0.35,
586            label='有储能', color=SEMANTIC['storage'], alpha=0.7, linewidth=0.2)
587     setup_ax(ax, '场景', '可再生利用率', '可再生利用率')
588     ax.legend(fontsize=7)
589     ax.tick_params(axis='x', labelsize=7)
590
591     # fig.suptitle(title, fontsize=8, fontweight='bold', y=1.01)
592     plt.tight_layout(pad=0.4)
593     if save_path:
594         save_fig(fig, save_path)
595     return fig
596
597 def plot_offgrid_power_balance(hours, P_wind, P_pv, P_conv, P_prod, P_curtail,
598                                title='离网运行功率平衡', save_path=None):
599     fig, axes = plt.subplots(2, 2, figsize=(7.2, 4.8))
600
601     ax = axes[0, 0]
602     ax.plot(hours, P_wind, 'o-', label='风电', color=SEMANTIC['wind'], linewidth=1, markersize=2.5)
603     ax.plot(hours, P_pv, 's-', label='光伏', color=SEMANTIC['solar'], linewidth=1, markersize=2.5)
604     ax.plot(hours, P_conv + P_prod, '-~', label='总负荷', color=SEMANTIC['load'], linewidth=1, markersize=2.5)
605     setup_ax(ax, '时刻/h', '功率/MW', '可再生出力与负荷')
606
607     ax = axes[0, 1]
608     ax.bar(hours, P_curtail, color=SEMANTIC['curtail'], alpha=0.8, width=0.7, linewidth=0.2)
609     setup_ax(ax, '时刻/h', '弃电功率/MW', '弃电量')
610
611     ax = axes[1, 0]
612     ax.fill_between(hours, 0, P_prod, label='制氢功率', color=SEMANTIC['alkel'], alpha=0.5)
613     ax.plot(hours, P_wind + P_pv - P_conv, '--', color='#999999', linewidth=0.8, label='可用功率')
614     setup_ax(ax, '时刻/h', '功率/MW', '制氢功率跟随可再生出力')
615
616     ax = axes[1, 1]
617     total_avail = np.sum(P_wind + P_pv - P_conv)
618     total_used = np.sum(P_prod)
619     total_curtailed = np.sum(P_curtail)
620
621     pie_vals = np.array([max(0, total_used), max(0, total_curtailed), max(0, -total_avail)])
622     pie_vals = pie_vals[pie_vals > 0]
623     pie_labels = ['自用', '弃电', '缺电'][:len(pie_vals)]
624     pie_colors = [SEMANTIC['all_pass'], SEMANTIC['curtail'], SEMANTIC['buy']][:len(pie_vals)]
625     wedges, texts, autotexts = ax.pie(pie_vals, labels=pie_labels,
626                                       autopct='%1.1f%%',
627                                       colors=pie_colors,
628                                       startangle=90, textprops={'fontsize': 7.5})
629     for t in autotexts:
630         t.set_fontsize(7)
631     ax.set_title('能量分配', fontsize=9, fontweight='bold')
632
633     # fig.suptitle(title, fontsize=8, fontweight='bold', y=1.01)
634     plt.tight_layout(pad=0.4)
635     if save_path:
636         save_fig(fig, save_path)
637     return fig

```

```

638
639 def plot_production_vs_curtailement(scenario_labels, productions, curtail_rates,
640                                     title='各场景产量与弃电分析', save_path=None):
641     fig, ax1 = plt.subplots(figsize=(7.2, 2.8))
642
643     sort_idx = np.argsort(curtail_rates)[::-1]
644     labels_sorted = [scenario_labels[i] for i in sort_idx]
645     prods_sorted = [productions[i] for i in sort_idx]
646     curtails_sorted = [curtail_rates[i] for i in sort_idx]
647
648     x = np.arange(len(labels_sorted))
649     bars = ax1.bar(x, prods_sorted, 0.6, label='日产量', color=SEMANTIC['alkel'], alpha=0.8, linewidth=0.2)
650     ax1.set_ylabel('日产量/(t/d)', fontsize=8.5, color=SEMANTIC['alkel'])
651     ax1.tick_params(axis='y', labelcolor=SEMANTIC['alkel'], labelsz=8)
652
653     ax2 = ax1.twinx()
654     ax2.plot(x, curtails_sorted, 'o-', color=SEMANTIC['curtail'], linewidth=1.2,
655             markersize=3.5, label='弃电率')
656     ax2.axhline(y=0.2, color=SEMANTIC['no_pass'], linestyle='--', linewidth=0.6, alpha=0.5)
657     ax2.set_ylabel('弃电率', fontsize=8.5, color=SEMANTIC['curtail'])
658     ax2.tick_params(axis='y', labelcolor=SEMANTIC['curtail'], labelsz=8)
659
660     ax1.set_xticks(x)
661     ax1.set_xticklabels(labels_sorted, fontsize=7, rotation=45, ha='right')
662     ax1.set_xlabel('场景', fontsize=8.5)
663
664     lines1, labels1 = ax1.get_legend_handles_labels()
665     lines2, labels2 = ax2.get_legend_handles_labels()
666     ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper right', fontsize=7.5)
667
668     # ax1.set_title(title, fontsize=7.5, fontweight='bold')
669     ax1.grid(True, alpha=0.12, axis='y')
670
671     plt.tight_layout()
672     if save_path:
673         save_fig(fig, save_path)
674     return fig
675
676 def plot_net_vs_offgrid_comparison(comparison_data, title='离网与联网模式经济性对比',
677                                    save_path=None):
678     metrics = ['吨氨成本', '净电费', '年产量', '能自给率']
679     fig, ax = plt.subplots(figsize=(5, 2.8))
680
681     x = np.arange(len(metrics))
682     w = 0.35
683
684     ongrid_vals = [comparison_data['ongrid'].get(m, 0) for m in metrics]
685     offgrid_vals = [comparison_data['offgrid'].get(m, 0) for m in metrics]
686
687     bars1 = ax.bar(x - w / 2, ongrid_vals, w, label='联网模式', color=SEMANTIC['wind'], alpha=0.8, linewidth=0.2)
688     bars2 = ax.bar(x + w / 2, offgrid_vals, w, label='离网模式', color=SEMANTIC['storage'], alpha=0.8, linewidth=0.2)
689
690     for i in range(len(metrics)):
691         ratio = offgrid_vals[i] / ongrid_vals[i] * 100 if ongrid_vals[i] != 0 else 0
692         ax.text(i, max(ongrid_vals[i], offgrid_vals[i]) * 1.05,
693               f'{ratio:.0f}%', ha='center', fontsize=7.5, fontweight='bold')
694
695     ax.set_xticks(x)
696     ax.set_xticklabels(metrics, fontsize=8)
697     setup_ax(ax, '', '')
698     ax.legend(fontsize=7)
699
700     # fig.suptitle(title, fontsize=7.5, fontweight='bold', y=1.02)
701     plt.tight_layout()
702     if save_path:
703         save_fig(fig, save_path)
704     return fig
705
706 def plot_policy_analysis(save_path=None):
707     # single column width
708     fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.2))
709
710     ax = axes[0]
711     impacts = ['调峰压力增大', '备用容量需求增加', '电能质量挑战',
712              '促进新能源消纳', '降低系统碳排放', '推动储能技术发展']
713     directions = [-1, -1, -1, 1, 1, 1]
714     magnitudes = [0.85, 0.7, 0.6, 0.9, 0.95, 0.75]
715     colors_impact = ['#E74C3C' if d < 0 else '#27AE60' for d in directions]
716     y_pos = np.arange(len(impacts))
717
718     ax.barh(y_pos, [d * m for d, m in zip(directions, magnitudes)],
719            color=colors_impact, alpha=0.8, height=0.5, linewidth=0.2)
720     for i, (imp, d, m) in enumerate(zip(impacts, directions, magnitudes)):
721         label = f'{imp} ({abs(m):.0%})'
722         ax.text(0.03 if d > 0 else -0.03, i, label,
723               ha='left' if d > 0 else 'right', va='center',
724               fontsize=7, fontweight='bold')
725     ax.axvline(x=0, color='black', linewidth=0.5)
726     ax.set_yticks(y_pos)
727     ax.set_yticklabels([])
728     ax.set_xlim(-1.3, 1.3)
729     ax.set_title('高渗透率对电网影响', fontsize=8.5, fontweight='bold')
730     ax.text(-1.2, -0.5, '← 不利影响', fontsize=7, color='#E74C3C', fontweight='bold')
731     ax.text(1.1, -0.5, '→ 有利影响', fontsize=7, color='#27AE60', fontweight='bold')
732     ax.grid(True, alpha=0.12, axis='x')
733     ax.tick_params(labelsz=8)
734

```

```

735 ax = axes[1]
736 policies = ['完善市场机制', '配置储能', '智能调度', '政策激励', '技术研发']
737 values = [0.88, 0.82, 0.75, 0.92, 0.78]
738 bars = ax.barh(policies, values, color='#2980B9', alpha=0.8, height=0.5, linewidth=0.2)
739 for i, v in enumerate(values):
740     ax.text(v + 0.02, i, f'{v:.0%}', va='center', fontsize=7.5, fontweight='bold')
741 ax.set_xlim(0, 1.1)
742 ax.set_title('政策建议框架', fontsize=8.5, fontweight='bold')
743 ax.grid(True, alpha=0.12, axis='x')
744 ax.tick_params(labelsize=8)
745
746 # fig.suptitle('绿电直连园区发展影响与政策建议', fontsize=8, fontweight='bold', y=1.02)
747 plt.tight_layout(pad=0.4)
748 if save_path:
749     save_fig(fig, save_path)
750 return fig
751
752 def plot_compliance_comparison(production_levels, q2_counts, q3_counts,
753                               title='Q2与Q3达标统计对比', save_path=None):
754     fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.8), sharey=True)
755
756     for idx, (ax, counts, label) in enumerate(zip(
757         axes, [q2_counts, q3_counts], ['离散调节(Q2)', '连续调节(Q3)'])):
758         x = np.arange(len(production_levels))
759         w = 0.5
760         bottom = np.zeros(len(production_levels))
761         for key, color in [(('all_pass', SEMANTIC['all_pass']),
762                             ('partial_pass', SEMANTIC['partial_pass']),
763                             ('no_pass', SEMANTIC['no_pass']))]:
764             vals = [counts[l][key] for l in production_levels]
765             bars = ax.bar(x, vals, w, bottom=bottom, color=color, alpha=0.8,
766                          edgecolor='white', linewidth=0.3, label={('all_pass': '全满足',
767                                                                      'partial_pass': '部分满足',
768                                                                      'no_pass': '全不满足')[key])
769             for i, v in enumerate(vals):
770                 if v > 0:
771                     ax.text(i, bottom[i] + v / 2, str(v), ha='center',
772                             va='center', fontsize=8, fontweight='bold', color='white')
773             bottom += vals
774         ax.set_xticks(x)
775         ax.set_xticklabels([f'{l} t/d' for l in production_levels], fontsize=7.5)
776         ax.set_title(label, fontsize=8.5, fontweight='bold')
777         ax.set_xlabel('日产量', fontsize=8)
778         ax.tick_params(labelsize=7.5)
779         if idx == 0:
780             ax.set_ylabel('场景数量', fontsize=8)
781
782     axes[0].legend(fontsize=7)
783     # fig.suptitle(title, fontsize=7.5, fontweight='bold', y=1.02)
784     plt.tight_layout(pad=0.4)
785     if save_path:
786         save_fig(fig, save_path)
787     return fig
788
789 def plot_q3_power_schedule(hours, P_wind, P_pv, P_conv, P_prod_continuous,
790                           P_prod_discrete, P_buy, P_sell, title='连续调节功率调度方案',
791                           save_path=None):
792     fig, axes = plt.subplots(2, 2, figsize=(7.2, 4.8))
793
794     ax = axes[0, 0]
795     ax.fill_between(hours, 0, P_prod_continuous, label='连续制氢功率',
796                    color=SEMANTIC['alkel'], alpha=0.45)
797     ax.plot(hours, P_wind + P_pv - P_conv, '--', color='#999999', linewidth=0.8,
798            label='可用功率')
799     setup_ax(ax, '时刻/h', '功率/MW', '连续可调功率方案')
800
801     ax = axes[0, 1]
802     ax.step(hours, P_prod_discrete, where='mid', label='离散(Q2)', color=SEMANTIC['load'],
803            linewidth=1.2)
804     ax.fill_between(hours, 0, P_prod_continuous, label='连续(Q3)',
805                    color=SEMANTIC['wind'], alpha=0.3)
806     setup_ax(ax, '时刻/h', '功率/MW', '连续和离散调度对比')
807
808     ax = axes[1, 0]
809     ax.bar(hours, P_buy, label='购电', color=SEMANTIC['buy'], alpha=0.8, width=0.7, linewidth=0.2)
810     ax.bar(hours, -P_sell, label='售电', color=SEMANTIC['sell'], alpha=0.8, width=0.7, linewidth=0.2)
811     ax.axhline(y=0, color='black', linewidth=0.4)
812     setup_ax(ax, '时刻/h', '功率/MW', '购电与售电')
813
814     ax = axes[1, 1]
815     equiv_load = P_conv + P_prod_continuous
816     ax.stackplot(hours, [P_wind, P_pv], labels=['风电', '光伏'],
817                  colors=[SEMANTIC['wind'], SEMANTIC['solar']], alpha=0.65)
818     ax.plot(hours, equiv_load, '-', color=SEMANTIC['load'], linewidth=1, label='等效总负荷')
819     setup_ax(ax, '时刻/h', '功率/MW', '可再生发电与等效负荷')
820
821     # fig.suptitle(title, fontsize=8, fontweight='bold', y=1.01)
822     plt.tight_layout(pad=0.4)
823     if save_path:
824         save_fig(fig, save_path)
825     return fig

```


q1_simulation.py

```
1 import numpy as np
2 import os
3 import sys
4 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
5 from config import Config, TIME_LABELS, FIG_DIR, TBL_DIR, HOURS
6 from utils import (power_balance, calc_daily_energy, calc_green_indicators,
7                   calc_electricity_cost, calc_ammonia_cost, calc_generation_cost)
8 from visualization import (plot_power_balance, plot_green_indicators_radar,
9                           plot_sankey_diagram, save_fig)
10
11 def run_q1():
12     cfg = Config()
13     Q_NH3 = 36.0
14     prod = cfg.get_production_power(Q_NH3)
15     P_prod = np.full(24, prod['total'])
16
17     P_wind = cfg.P_wind_typ
18     P_pv = cfg.P_pv_typ
19     P_conv = cfg.P_conv
20
21     P_buy, P_sell, P_re = power_balance(P_wind, P_pv, P_conv, P_prod)
22
23     E_re, E_total, E_sell, E_buy, E_self = calc_daily_energy(
24         P_wind, P_pv, P_conv, P_prod, P_buy, P_sell)
25
26     indicators = calc_green_indicators(E_re, E_total, E_sell, E_buy, cfg)
27
28     net_elec_cost = calc_electricity_cost(P_buy, P_sell, cfg.price_buy, cfg.price_sell)
29     gen_cost = calc_generation_cost(cfg)
30     total_cost, cost_per_ton = calc_ammonia_cost(
31         net_elec_cost + gen_cost, Q_NH3, cfg, Q_NH3)
32
33     results = {
34         'P_wind': P_wind, 'P_pv': P_pv, 'P_conv': P_conv,
35         'P_prod': P_prod, 'P_buy': P_buy, 'P_sell': P_sell,
36         'E_re': E_re, 'E_total': E_total, 'E_sell': E_sell,
37         'E_buy': E_buy, 'E_self': E_self,
38         'indicators': indicators,
39         'net_elec_cost': net_elec_cost,
40         'gen_cost': gen_cost,
41         'total_cost': total_cost,
42         'cost_per_ton': cost_per_ton,
43         'nh3_production': Q_NH3,
44         'P_re': P_re,
45     }
46
47     return results
48
49 def plot_q1_results(results, cfg):
50     fig1 = plot_power_balance(
51         HOURS, results['P_wind'], results['P_pv'], results['P_conv'],
52         results['P_buy'], results['P_sell'], results['P_prod'],
53         title='园区典型日功率平衡曲线')
54     save_fig(fig1, 'fig1_power_balance.pdf', 'q1')
55
56     radar_data = {
57         '自发自用比例': results['indicators']['self_consume_ratio'],
58         '总用电绿电比例': results['indicators']['green_ratio'],
59         '上网电量比例': results['indicators']['curtail_ratio'],
60     }
61     radar_req = {
62         '自发自用比例': cfg.REQUIRE_SELF_CONSUME,
63         '总用电绿电比例': cfg.REQUIRE_GREEN_RATIO,
64         '上网电量比例': cfg.REQUIRE_CURTAIL_RATIO,
65     }
66     fig2 = plot_green_indicators_radar(
67         radar_data, radar_req, title='绿电指标达标分析')
68     save_fig(fig2, 'fig2_green_indicators_radar.pdf', 'q1')
69
70     fig3 = plot_sankey_diagram(
71         values=[results['E_re'] - results['E_sell'], results['E_sell'],
72               results['E_buy'], results['E_self'], results['E_sell'] * 0.5, results['E_buy'] * 0.5],
73         labels=['自用可再生', '上网电量', '网购电量', '制氢氨', '售电(收入)', '购电(支出)'],
74         title='园区日能量流向')
75     save_fig(fig3, 'fig3_sankey.pdf', 'q1')
76
77 def print_q1_results(results, cfg):
78     ind = results['indicators']
79     print('=' * 60)
80     print('问题一：典型风光场景分析结果')
81     print('=' * 60)
82     print(f'新能源发电量:      {results["E_re"]:.2f} MWh')
83     print(f'总用电量:      {results["E_total"]:.2f} MWh')
84     print(f'上网电量:      {results["E_sell"]:.2f} MWh')
85     print(f'网购电量:      {results["E_buy"]:.2f} MWh')
86     print(f'新能源自发自用电量: {results["E_self"]:.2f} MWh')
87     print()
88     print(f'自发自用比例:      {ind["self_consume_ratio"]:.2%} (要求>60%) {" " if ind["self_consume_pass"] else " "}')
89     print(f'总用电绿电比例:      {ind["green_ratio"]:.2%} (要求>30%) {" " if ind["green_ratio_pass"] else " "}')
90     print(f'上网电量比例:      {ind["curtail_ratio"]:.2%} (要求<20%) {" " if ind["curtail_pass"] else " "}')
91     print()
92     print(f'净电费:      {results["net_elec_cost"]:.2f} 元')
93     print(f'发电成本:      {results["gen_cost"]:.2f} 元')
```



```

94     print(f'吨氨成本:           {results["cost_per_ton"]:.2f} 元/吨')
95     print('=' * 60)
96
97 if __name__ == '__main__':
98     cfg = Config()
99     results = run_q1()
100    print_q1_results(results, cfg)
101    plot_q1_results(results, cfg)
102    print('Q1 figures saved.')

```

q2_scheduling.py

```

1  import numpy as np
2  import os, sys
3  sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
4  from config import Config, HOURS, FIG_DIR, TBL_DIR
5  from utils import (power_balance, calc_daily_energy, calc_green_indicators,
6                    calc_electricity_cost, calc_ammonia_cost, calc_generation_cost,
7                    calc_om_cost, classify_scenario_indicators,
8                    generate_24_scenarios, level_to_capacity)
9  from visualization import (plot_gantt_chart, plot_cost_vs_production,
10                            plot_24_scenarios_heatmap, plot_scenario_compliance,
11                            plot_annual_cost_distribution, save_fig)
12
13 def optimize_schedule(P_wind, P_pv, P_conv, P_prod, cfg, k_hours):
14     T = 24
15     marginal_costs = np.zeros(T)
16     om_hourly = (cfg.alkel_om * 20 + cfg.pemel_om * 20 + cfg.nh3_om * 1.5)
17
18     for t in range(T):
19         net_off = P_wind[t] + P_pv[t] - P_conv[t]
20         if net_off >= 0:
21             cost_off = -net_off * cfg.price_sell
22         else:
23             cost_off = -net_off * cfg.price_buy[t]
24
25         net_on = net_off - P_prod
26         if net_on >= 0:
27             cost_on = -net_on * cfg.price_sell
28         else:
29             cost_on = -net_on * cfg.price_buy[t]
30
31         marginal_costs[t] = cost_on + om_hourly - cost_off
32
33     sorted_idx = np.argsort(marginal_costs)
34     selected = np.zeros(T, dtype=bool)
35     selected[sorted_idx[:k_hours]] = True
36
37     P_prod_sched = np.where(selected, P_prod, 0.0)
38     P_buy, P_sell, P_re = power_balance(P_wind, P_pv, P_conv, P_prod_sched)
39
40     return selected, P_buy, P_sell, marginal_costs
41
42 def analyze_production_level(cfg, scenario, target_tpd, installed_capacity=72):
43     k_hours = int(round(target_tpd / installed_capacity * 24))
44     if k_hours == 0:
45         return None
46
47     prod = cfg.get_production_power(installed_capacity)
48     P_prod = prod['total']
49     nh3_rate = prod['nh3_rate_th']
50
51     P_wind = scenario['P_wind']
52     P_pv = scenario['P_pv']
53     P_conv = cfg.P_conv
54
55     selected, P_buy, P_sell, _ = optimize_schedule(
56         P_wind, P_pv, P_conv, P_prod, cfg, k_hours)
57
58     P_prod_sched = np.where(selected, P_prod, 0.0)
59     E_re, E_total, E_sell, E_buy, E_self = calc_daily_energy(
60         P_wind, P_pv, P_conv, P_prod_sched, P_buy, P_sell)
61
62     indicators = calc_green_indicators(E_re, E_total, E_sell, E_buy, cfg)
63
64     net_elec_cost = calc_electricity_cost(P_buy, P_sell, cfg.price_buy, cfg.price_sell)
65     gen_cost = calc_generation_cost(cfg)
66
67     hours_run = np.sum(selected)
68     nh3_production = hours_run * nh3_rate
69
70     om_cost = calc_om_cost(nh3_production, cfg, installed_capacity)
71
72     total_cost = net_elec_cost + gen_cost + om_cost
73     cost_per_ton = total_cost / nh3_production if nh3_production > 0 else 0
74     util_rate = hours_run / 24.0
75
76     return {
77         'capacity': target_tpd,
78         'installed_capacity': installed_capacity,
79         'nh3_production': nh3_production,

```



```

177 fig4 = plot_gantt_chart(
178     cfg, [q2_data['sched_data'][i] for i in range(len(levels))],
179     levels, title='各产量最优生产时段安排（典型场景）')
180 save_fig(fig4, 'fig4_gantt.pdf', 'q2')
181
182 fig5 = plot_cost_vs_production(
183     levels, [typ_results[l] for l in levels],
184     title='关键指标随产量变化（典型场景）')
185 save_fig(fig5, 'fig5_cost_vs_production.pdf', 'q2')
186
187 scenario_ids = [s['label'] for s in scenarios]
188 cost_matrix = np.zeros((len(scenarios), len(levels)))
189 for i in range(len(scenarios)):
190     for j, l in enumerate(levels):
191         if i < len(all_results[l]):
192             cost_matrix[i, j] = all_results[l][i]['cost_per_ton']
193 fig6 = plot_24_scenarios_heatmap(
194     scenario_ids, levels, cost_matrix,
195     title='24场景×5产量吨氨成本热力图')
196 save_fig(fig6, 'fig6_heatmap.pdf', 'q2')
197
198 fig7 = plot_scenario_compliance(
199     compliance_counts, levels,
200     title='各产量绿电指标达标统计（24场景）')
201 save_fig(fig7, 'fig7_compliance.pdf', 'q2')
202
203 fig8 = plot_annual_cost_distribution(
204     q2_data['annual_costs'], levels,
205     title='全年吨氨成本分布曲线（离散调节）')
206 save_fig(fig8, 'fig8_annual_cost_distribution.pdf', 'q2')
207
208 if __name__ == '__main__':
209     cfg = Config()
210     q2_data = run_q2()
211     plot_q2_results(q2_data, cfg)
212     print('\nQ2 figures saved.')
213

```

q3_continuous.py

```

1 import numpy as np
2 import os, sys
3 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
4 from config import Config, HOURS, FIG_DIR, TBL_DIR
5 from utils import (power_balance, calc_daily_energy, calc_green_indicators,
6                   calc_electricity_cost, calc_generation_cost, calc_om_cost,
7                   classify_scenario_indicators, generate_24_scenarios,
8                   level_to_capacity)
9 from visualization import (plot_q3_power_schedule, plot_comparison_q2_q3,
10                           plot_compliance_comparison, plot_annual_cost_distribution,
11                           save_fig)
12
13 def optimize_continuous(P_wind, P_pv, P_conv, cfg, target_nh3_t, P_max, P_min_ratio=0.1):
14     T = 24
15     prod_cfg = cfg.get_production_power(72)
16     P_prod_max = prod_cfg['total']
17     P_prod_min = P_prod_max * P_min_ratio
18     nh3_per_mwh = prod_cfg['nh3_rate_th'] / P_prod_max
19     total_energy_needed = target_nh3_t / nh3_per_mwh
20     om_per_mwh = (cfg.alkel_om * 0.5 + cfg.pemel_om * 0.5 + cfg.nh3_om * 0.5)
21
22     net_avail = P_wind + P_pv - P_conv
23
24     p_opt = np.zeros(T)
25     remaining = total_energy_needed
26
27     surplus_hours = np.argsort(-net_avail)
28     for t in surplus_hours:
29         if remaining <= 0:
30             break
31         if net_avail[t] <= 0:
32             break
33         use = min(remaining, net_avail[t], P_prod_max)
34         if use >= P_prod_min or (use > 0 and remaining <= use):
35             p_opt[t] = use
36             remaining -= use
37
38     if remaining > 0:
39         class HourCost:
40             def __init__(self, idx, net):
41                 self.idx = idx
42                 if net >= 0:
43                     self.cost = -cfg.price_sell + om_per_mwh
44                 else:
45                     self.cost = cfg.price_buy[idx] + om_per_mwh
46                 self.avail = P_prod_max - p_opt[idx]
47
48         sorted_hours = sorted([HourCost(t, net_avail[t]) for t in range(T)],
49                               key=lambda h: h.cost)
50
51         for h in sorted_hours:

```

```

52         if remaining <= 0:
53             break
54         add = min(remaining, h.avail)
55         if add >= P_prod_min or p_opt[h.idx] > 0 or remaining >= P_prod_min:
56             p_opt[h.idx] += add
57             remaining -= add
58
59     if remaining > 0.01:
60         running_hours = [t for t in range(T) if p_opt[t] >= P_prod_min * 0.5]
61         if running_hours:
62             running_hours.sort(key=lambda t: P_prod_max - p_opt[t], reverse=True)
63             for t in running_hours:
64                 if remaining <= 0.01:
65                     break
66                 add = min(remaining, P_prod_max - p_opt[t])
67                 p_opt[t] += add
68                 remaining -= add
69
70     if remaining > 0.01:
71         candidates = [(t, P_prod_max - p_opt[t], net_avail[t]) for t in range(T)]
72         candidates.sort(key=lambda x: -x[2])
73         t, cap, _ = candidates[0]
74         add = min(remaining, cap)
75         p_opt[t] += add
76         remaining -= add
77
78     if remaining > 0.01:
79         p_opt[np.argmax(P_prod_max - p_opt)] += remaining
80
81     p_opt = np.clip(p_opt, 0, P_prod_max)
82     running = p_opt >= P_prod_min * 0.5
83     if np.any(p_opt > 0) and not np.any(running):
84         max_idx = np.argmax(p_opt)
85         running[max_idx] = True
86     p_opt[-running] = 0
87
88     nh3_produced = np.sum(p_opt) * nh3_per_mwh
89     return p_opt, nh3_produced
90
91 def analyze_continuous_production(cfg, scenario, target_nh3, capacity_tpd=72):
92     P_wind = scenario['P_wind']
93     P_pv = scenario['P_pv']
94     P_conv = cfg.P_conv
95     prod_cfg = cfg.get_production_power(capacity_tpd)
96     P_prod_max = prod_cfg['total']
97
98     p_opt, nh3_produced = optimize_continuous(
99         P_wind, P_pv, P_conv, cfg, target_nh3, P_prod_max)
100
101     P_buy, P_sell, P_re = power_balance(P_wind, P_pv, P_conv, p_opt)
102     E_re, E_total, E_sell, E_buy, E_self = calc_daily_energy(
103         P_wind, P_pv, P_conv, p_opt, P_buy, P_sell)
104
105     indicators = calc_green_indicators(E_re, E_total, E_sell, E_buy, cfg)
106     net_elec_cost = calc_electricity_cost(P_buy, P_sell, cfg.price_buy, cfg.price_sell)
107     gen_cost = calc_generation_cost(cfg)
108     om_cost = calc_om_cost(nh3_produced, cfg, capacity_tpd)
109
110     total_cost = net_elec_cost + gen_cost + om_cost
111     cost_per_ton = total_cost / nh3_produced if nh3_produced > 0 else 0
112
113     return {
114         'capacity': target_nh3,
115         'nh3_production': nh3_produced,
116         'p_opt': p_opt,
117         'indicators': indicators,
118         'net_elec_cost': net_elec_cost,
119         'gen_cost': gen_cost,
120         'om_cost': om_cost,
121         'dep_cost': 0,
122         'total_cost': total_cost,
123         'cost_per_ton': cost_per_ton,
124         'E_re': E_re, 'E_total': E_total, 'E_sell': E_sell, 'E_buy': E_buy,
125         'P_buy': P_buy, 'P_sell': P_sell,
126         'classification': classify_scenario_indicators(indicators),
127     }
128
129 def run_q3():
130     cfg = Config()
131     production_levels = [72, 63, 54, 45, 36]
132     scenarios = generate_24_scenarios(cfg)
133
134     typical_scenario = {
135         'P_wind': cfg.P_wind_typ,
136         'P_pv': cfg.P_pv_typ,
137     }
138
139     print('=' * 70)
140     print('问题三：连续调节制氮优化结果')
141     print('=' * 70)
142
143     print('\n--- 典型场景连续调节结果 ---')
144     q3_typical = {}
145     for cap in production_levels:
146         res = analyze_continuous_production(cfg, typical_scenario, cap)
147         q3_typical[cap] = res
148         ind = res['indicators']
149         print(f' 目标 {cap:2d} t/d | 实产 {res["nh3_production"]:.1f} t | ')

```

```

150         f'成本 {res["cost_per_ton"]:.8f} 元 | '
151         f'自用/绿电/上网 {ind["self_consume_ratio"]:.1%}/{ind["green_ratio"]:.1%}/{ind["curtail_ratio"]:.1%}')
152
153     print('\n--- 24场景分析 ---')
154     q3_scenario = {}
155     for cap in production_levels:
156         q3_scenario[cap] = []
157         for scen in scenarios:
158             res = analyze_continuous_production(cfg, scen, cap)
159             if res and res['nh3_production'] > 0:
160                 q3_scenario[cap].append(res)
161
162     compliance_counts = {}
163     for cap in production_levels:
164         counts = {'all_pass': 0, 'partial_pass': 0, 'no_pass': 0}
165         for r in q3_scenario[cap]:
166             counts[r['classification']] += 1
167         compliance_counts[cap] = counts
168
169     print(f'{"目标产量":>8s} | {"全满足":>8s} | {"部分满足":>8s} | {"全不满足":>8s}')
170     for cap in production_levels:
171         c = compliance_counts[cap]
172         print(f'{"cap":5d} t/d | {c["all_pass"]:6d} | {c["partial_pass"]:6d} | {c["no_pass"]:6d}')
173
174     annual_costs = {}
175     for cap in production_levels:
176         costs = [r['cost_per_ton'] for r in q3_scenario[cap] if r['cost_per_ton'] > 0]
177         annual_costs[cap] = costs
178
179     return {
180         'q3_typical': q3_typical,
181         'q3_scenario': q3_scenario,
182         'compliance_counts': compliance_counts,
183         'annual_costs': annual_costs,
184         'production_levels': production_levels,
185         'scenarios': scenarios,
186         'typical_scenario': typical_scenario,
187     }
188
189 def plot_q3_results(q3_data, cfg, q2_data=None):
190     levels = q3_data['production_levels']
191     typ_results = q3_data['q3_typical']
192
193     first_cap = levels[0]
194     first_res = typ_results[first_cap]
195     if first_res:
196         P_wind = cfg.P_wind_typ if 'P_wind' in q3_data['typical_scenario'] else cfg.P_wind_typ
197         P_pv = cfg.P_pv_typ
198         plot_input_data = {
199             'P_wind': P_wind,
200             'P_pv': P_pv,
201             'P_conv': cfg.P_conv,
202             'P_prod_continuous': first_res['p_opt'],
203             'P_prod_discrete': q2_data['sched_data'][0] * cfg.get_production_power(levels[0])['total'] if q2_data else np.zeros(24),
204             'P_buy': first_res.get('P_buy', np.zeros(24)),
205             'P_sell': first_res.get('P_sell', np.zeros(24)),
206         }
207
208     sched = q2_data['sched_data'][0] if q2_data else np.zeros(24, dtype=bool)
209     prod_power = cfg.get_production_power(levels[0])['total']
210     fig9 = plot_q3_power_schedule(
211         HOURS, plot_input_data['P_wind'], plot_input_data['P_pv'],
212         plot_input_data['P_conv'], plot_input_data['P_prod_continuous'],
213         sched * prod_power, plot_input_data['P_buy'], plot_input_data['P_sell'],
214         title=f'连续调节功率调度方案 ({levels[0]} t/d 典型场景)')
215     save_fig(fig9, 'fig9_q3_power_schedule.pdf', 'q3')
216
217     if q2_data:
218         q3_costs = [typ_results[l]['cost_per_ton'] for l in levels]
219         q2_costs = [q2_data['typical_results'][l]['cost_per_ton'] for l in levels]
220         q3_inds = {l: typ_results[l]['indicators'] for l in levels}
221         q2_inds = {l: q2_data['typical_results'][l]['indicators'] for l in levels}
222
223     fig10 = plot_comparison_q2_q3(
224         levels, q2_costs, q3_costs, q2_inds, q3_inds,
225         title='Q2 vs Q3 吨氨成本与绿电指标对比 (典型场景)')
226     save_fig(fig10, 'fig10_q2_vs_q3.pdf', 'q3')
227
228     fig11 = plot_annual_cost_distribution(
229         q3_data['annual_costs'], levels,
230         title='全年吨氨成本分布曲线 (连续调节)')
231     save_fig(fig11, 'fig11_q3_annual_distribution.pdf', 'q3')
232
233     fig12 = plot_compliance_comparison(
234         levels, q2_data['compliance_counts'], q3_data['compliance_counts'],
235         title='Q2 vs Q3 绿电指标达标统计对比')
236     save_fig(fig12, 'fig12_q3_vs_q2_compliance.pdf', 'q3')
237
238 if __name__ == '__main__':
239     cfg = Config()
240     q3_data = run_q3()
241     try:
242         from q2_scheduling import run_q2
243         q2_data = run_q2()
244         plot_q3_results(q3_data, cfg, q2_data)
245     except:
246         plot_q3_results(q3_data, cfg)

```

```
247 print('\nQ3 figures saved.')
```

q4_offgrid.py

```
1 import numpy as np
2 import os, sys
3 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
4 from config import Config, HOURS, FIG_DIR, TBL_DIR
5 from utils import (power_balance, calc_daily_energy, calc_green_indicators,
6                   calc_electricity_cost, calc_generation_cost, calc_om_cost,
7                   classify_scenario_indicators, generate_24_scenarios)
8 from visualization import (plot_offgrid_power_balance, plot_production_vs_curtailement,
9                             plot_pareto_storage, plot_soc_discharge,
10                             plot_offgrid_comparison, plot_net_vs_offgrid_comparison,
11                             save_fig)
12
13 def offgrid_without_storage(P_wind, P_pv, P_conv, cfg, P_prod_max, P_prod_min):
14     T = 24
15     p_prod = np.zeros(T)
16     P_curtail = np.zeros(T)
17
18     for t in range(T):
19         avail = P_wind[t] + P_pv[t] - P_conv[t]
20         if avail >= P_prod_max:
21             p_prod[t] = P_prod_max
22             P_curtail[t] = avail - P_prod_max
23         elif avail >= P_prod_min:
24             p_prod[t] = avail
25         elif avail > 0:
26             p_prod[t] = 0
27             P_curtail[t] = avail
28         else:
29             p_prod[t] = 0
30
31     nh3_per_mwh = cfg.get_production_power(72)['nh3_rate_th'] / P_prod_max
32     nh3_produced = np.sum(p_prod) * nh3_per_mwh
33     total_curtail = np.sum(P_curtail)
34     re_gen = np.sum(P_wind + P_pv)
35     re_util = (re_gen - total_curtail) / re_gen if re_gen > 0 else 0
36
37     return p_prod, P_curtail, nh3_produced, total_curtail, re_util
38
39 def offgrid_with_storage(P_wind, P_pv, P_conv, cfg, P_prod_max, P_prod_min,
40                          storage_cap_MWh, storage_power_MW):
41     T = 24
42     p_prod = np.zeros(T)
43     P_curtail = np.zeros(T)
44     soc = np.zeros(T + 1)
45     P_ch = np.zeros(T)
46     P_dis = np.zeros(T)
47
48     ch_eff = cfg.storage_ch_eff
49     dis_eff = cfg.storage_dis_eff
50     self_disch = cfg.storage_self_discharge
51     soc[0] = storage_cap_MWh * 0.2
52
53     for t in range(T):
54         net_before = P_wind[t] + P_pv[t] - P_conv[t]
55
56         if net_before >= P_prod_min:
57             p_prod[t] = min(net_before, P_prod_max)
58             leftover = net_before - p_prod[t]
59             if leftover > 0 and soc[t] < storage_cap_MWh:
60                 ch_max = min(storage_power_MW, (storage_cap_MWh - soc[t]) / ch_eff)
61                 P_ch[t] = min(leftover, ch_max)
62                 if leftover - P_ch[t] > 0:
63                     P_curtail[t] = leftover - P_ch[t]
64             else:
65                 needed = P_prod_min - net_before if net_before < P_prod_min else 0
66                 avail_dis = min(storage_power_MW, soc[t] * dis_eff)
67                 P_dis[t] = min(needed, avail_dis) if net_before < 0 else 0
68
69                 if net_before <= 0 and net_before + P_dis[t] >= 0:
70                     p_prod[t] = min(P_prod_max, P_dis[t] + net_before)
71                     if p_prod[t] < P_prod_min:
72                         p_prod[t] = 0
73                 elif net_before > 0 and net_before < P_prod_min:
74                     p_prod[t] = 0
75                     leftover = net_before
76                     if soc[t] < storage_cap_MWh:
77                         ch_max = min(storage_power_MW, (storage_cap_MWh - soc[t]) / ch_eff)
78                         P_ch[t] = min(leftover, ch_max)
79                     if leftover - P_ch[t] > 0:
80                         P_curtail[t] = leftover - P_ch[t]
81
82                 soc[t + 1] = min(storage_cap_MWh,
83                                 soc[t] * (1 - self_disch) + P_ch[t] * ch_eff - P_dis[t] / dis_eff)
84                 soc[t + 1] = max(0, soc[t + 1])
85
86     nh3_per_mwh = cfg.get_production_power(72)['nh3_rate_th'] / P_prod_max
87     nh3_produced = np.sum(p_prod) * nh3_per_mwh
```

```

88     total_curtail = np.sum(P_curtail)
89     re_gen = np.sum(P_wind + P_pv)
90     re_util = (re_gen - total_curtail) / re_gen if re_gen > 0 else 0
91
92     return p_prod, P_curtail, soc, P_ch, P_dis, nh3_produced, total_curtail, re_util
93
94 def calc_offgrid_cost(nh3_produced, cfg, capacity_tpd=72, P_wind_actual=None, P_pv_actual=None):
95     if P_wind_actual is not None and P_pv_actual is not None:
96         gen_cost = np.sum(P_wind_actual) * cfg.wind_cost + np.sum(P_pv_actual) * cfg.pv_cost
97     else:
98         gen_cost = calc_generation_cost(cfg)
99     om_cost = calc_om_cost(nh3_produced, cfg, capacity_tpd)
100
101     total_cost = gen_cost + om_cost
102     cost_per_ton = total_cost / nh3_produced if nh3_produced > 0 else 0
103     return total_cost, cost_per_ton, gen_cost, om_cost, 0
104
105 def run_q4():
106     cfg = Config()
107     scenarios = generate_24_scenarios(cfg)
108     prod_cfg = cfg.get_production_power(72)
109     P_prod_max = prod_cfg['total']
110     P_prod_min = P_prod_max * 0.1
111
112     print('=' * 70)
113     print('问题四：离网运行分析及储能配置研究')
114     print('=' * 70)
115
116     print('\n--- 4.1 离网无储能运行 ---\n')
117     q4_1_results = []
118     for scen in scenarios:
119         p_prod, P_curtail, nh3_prod, total_curt, re_util = offgrid_without_storage(
120             scen['P_wind'], scen['P_pv'], cfg.P_conv, cfg, P_prod_max, P_prod_min)
121         total_cost, cost_per_ton, _, _, _ = calc_offgrid_cost(
122             nh3_prod, cfg, P_wind_actual=scen['P_wind'], P_pv_actual=scen['P_pv'])
123         avail_energy = np.sum(scen['P_wind'] + scen['P_pv']) - cfg.P_conv
124         self_suff = nh3_prod * (cfg.get_production_power(72)['nh3_rate_th'] / P_prod_max) / np.maximum(1, np.sum(p_prod > 0))
125         energy_self_suff = (np.sum(scen['P_wind'] + scen['P_pv']) - total_curt) / np.sum(scen['P_wind'] + scen['P_pv']) if
            np.sum(scen['P_wind'] + scen['P_pv']) > 0 else 0
126
127         q4_1_results.append({
128             'scenario': scen,
129             'p_prod': p_prod,
130             'P_curtail': P_curtail,
131             'nh3_produced': nh3_prod,
132             'total_curtail': total_curt,
133             're_util': re_util,
134             'cost_per_ton': cost_per_ton,
135             'total_cost': total_cost,
136             'self_suff': self_suff,
137             'energy_self_suff': energy_self_suff,
138         })
139
140     total_annual_nh3 = sum(r['nh3_produced'] for r in q4_1_results) * cfg.days_per_scenario
141     avg_cost = np.mean([r['cost_per_ton'] for r in q4_1_results if r['cost_per_ton'] > 0])
142     avg_util = np.mean([r['nh3_produced'] / 72.0 for r in q4_1_results])
143     avg_re_util = np.mean([r['re_util'] for r in q4_1_results])
144
145     print(f' 全年制氢总量: {total_annual_nh3:.0f} 吨')
146     print(f' 平均吨氢成本: {avg_cost:.0f} 元/吨')
147     print(f' 平均产能利用率: {avg_util:.1%}')
148     print(f' 平均可再生利用率: {avg_re_util:.1%}')
149
150     max_curtail_scenario = max(q4_1_results, key=lambda r: r['total_curtail'])
151     min_wind_scenario = min(q4_1_results, key=lambda r: r['nh3_produced'])
152
153     print(f'\n 最大弃电场景: {max_curtail_scenario["scenario"]["label"]}, '
154           f' 弃电: {max_curtail_scenario["total_curtail"]:.1f} MWh')
155     print(f' 最小产量场景: {min_wind_scenario["scenario"]["label"]}, '
156           f' 产量: {min_wind_scenario["nh3_produced"]:.1f} t')
157
158     print('\n--- 4.2 储能配置优化 ---\n')
159     storage_caps = np.arange(0, 201, 20)
160     storage_power = 20
161     best_storage_cap = 0
162     best_avg_cost = float('inf')
163
164     pareto_costs = []
165     pareto_curtails = []
166
167     for s_cap in storage_caps:
168         scenario_costs = []
169         scenario_curtails = []
170         for scen in scenarios:
171             p_prod, P_curtail, soc, P_ch, P_dis, nh3_prod, total_curt, re_util = \
172                 offgrid_with_storage(scen['P_wind'], scen['P_pv'], cfg.P_conv,
173                                     cfg, P_prod_max, P_prod_min, s_cap, storage_power)
174             total_cost, cost_per_ton, _, _, _ = calc_offgrid_cost(
175                 nh3_prod, cfg, P_wind_actual=scen['P_wind'], P_pv_actual=scen['P_pv'])
176
177             if s_cap > 0:
178                 storage_inv = s_cap * 1000 * cfg.storage_inv / (cfg.storage_life * 365)
179                 storage_om = s_cap * cfg.storage_om
180                 total_cost += storage_inv + storage_om
181                 cost_per_ton = total_cost / nh3_prod if nh3_prod > 0 else 0
182
183             scenario_costs.append(cost_per_ton)

```

```

184         scenario_curtails.append(total_curt / max(1, np.sum(scen['P_wind'] + scen['P_pv'])))
185
186     avg_scenario_cost = np.mean([c for c in scenario_costs if c > 0])
187     avg_curtail = np.mean(scenario_curtails)
188
189     pareto_costs.append(avg_scenario_cost)
190     pareto_curtails.append(avg_curtail)
191
192     if avg_scenario_cost < best_avg_cost:
193         best_avg_cost = avg_scenario_cost
194         best_storage_cap = s_cap
195
196 print(f' 最优储能容量: {best_storage_cap:.0f} MWh')
197 print(f' 最优下平均吨氨成本: {best_avg_cost:.0f} 元/吨')
198
199 q4_2_storage_results = []
200 for scen in scenarios:
201     p_prod, P_curtail, soc, P_ch, P_dis, nh3_prod, total_curt, re_util = \
202         offgrid_with_storage(scen['P_wind'], scen['P_pv'], cfg.P_conv,
203                             cfg.P_prod_max, P_prod_min,
204                             best_storage_cap, storage_power)
205     total_cost, cost_per_ton, _, _, _ = calc_offgrid_cost(
206         nh3_prod, cfg, P_wind_actual=scen['P_wind'], P_pv_actual=scen['P_pv'])
207     storage_inv = best_storage_cap * 1000 * cfg.storage_inv / (cfg.storage_life * 365) if best_storage_cap > 0 else 0
208     storage_om = best_storage_cap * cfg.storage_om
209     total_cost += storage_inv + storage_om
210     cost_per_ton = total_cost / nh3_prod if nh3_prod > 0 else 0
211
212     re_gen_total = np.sum(scen['P_wind'] + scen['P_pv'])
213     energy_self_suff = (re_gen_total - total_curt) / re_gen_total if re_gen_total > 0 else 0
214     q4_2_storage_results.append({
215         'scenario': scen,
216         'p_prod': p_prod,
217         'P_curtail': P_curtail,
218         'soc': soc,
219         'P_ch': P_ch,
220         'P_dis': P_dis,
221         'nh3_produced': nh3_prod,
222         'total_curtail': total_curt,
223         're_util': re_util,
224         'energy_self_suff': energy_self_suff,
225         'cost_per_ton': cost_per_ton,
226     })
227
228 total_annual_nh3_s = sum([r['nh3_produced'] for r in q4_2_storage_results]) * cfg.days_per_scenario
229 avg_cost_s = np.mean([r['cost_per_ton'] for r in q4_2_storage_results if r['cost_per_ton'] > 0])
230 avg_util_s = np.mean([r['nh3_produced'] / 72.0 for r in q4_2_storage_results])
231
232 print(f'\n 有储能全年制氨总量: {total_annual_nh3_s:.0f} 吨')
233 print(f' 有储水平均吨氨成本: {avg_cost_s:.0f} 元/吨')
234 print(f' 有储水平均产能利用率: {avg_util_s:.1%}')
235
236 print('\n--- 4.3 离网 vs 联网经济性对比 ---\n')
237
238 from q3_continuous import analyze_continuous_production as q3_func
239 q3_annual_total = 0
240 q3_costs = []
241 q3_re_utils = []
242 for i, scen in enumerate(scenarios):
243     offgrid_nh3 = q4_1_results[i]['nh3_produced']
244     target = max(offgrid_nh3, 0.1)
245     res = q3_func(cfg, scen, target)
246     actual_gen_cost = (np.sum(scen['P_wind']) * cfg.wind_cost +
247                     np.sum(scen['P_pv']) * cfg.pv_cost)
248     res['total_cost'] = res['total_cost'] - res['gen_cost'] + actual_gen_cost
249     res['gen_cost'] = actual_gen_cost
250     res['cost_per_ton'] = res['total_cost'] / res['nh3_production'] if res['nh3_production'] > 0 else 0
251     q3_annual_total += res['nh3_production'] * cfg.days_per_scenario
252     q3_costs.append(res['cost_per_ton'])
253     if res['E_re'] > 0:
254         q3_re_utils.append(1 - res['E_sell'] / res['E_re'])
255     else:
256         q3_re_utils.append(0)
257 q3_avg_cost = np.mean(q3_costs) if q3_costs else 0
258 q3_re_util = np.mean(q3_re_utils) if q3_re_utils else 0
259
260 comparison_data = {
261     'offgrid': {
262         '吨氨成本': avg_cost,
263         '净电费': avg_cost * total_annual_nh3 / 360,
264         '年产量': total_annual_nh3,
265         '能自给率': avg_re_util,
266     },
267     'offgrid_storage': {
268         '吨氨成本': avg_cost_s if best_storage_cap > 0 else avg_cost,
269         '净电费': (avg_cost_s if best_storage_cap > 0 else avg_cost) * (total_annual_nh3_s if best_storage_cap > 0 else
270             total_annual_nh3) / 360,
271         '年产量': total_annual_nh3_s if best_storage_cap > 0 else total_annual_nh3,
272         '能自给率': np.mean([r['energy_self_suff'] for r in q4_2_storage_results]),
273     },
274     'ongrid': {
275         '吨氨成本': q3_avg_cost,
276         '净电费': q3_avg_cost * q3_annual_total / 360,
277         '年产量': q3_annual_total,
278         '能自给率': q3_re_util,
279     },
280 }

```



```

280
281 print(f' 无储能离网: {avg_cost:.0f} 元/吨, {total_annual_nh3:.0f} 吨/年')
282 print(f' 有储能离网: {avg_cost_s:.0f} 元/吨, {total_annual_nh3_s:.0f} 吨/年')
283 print(f' 产量提升: {(total_annual_nh3_s/total_annual_nh3-1)*100:.1f}%')
284 print(f' 联网对比(同等产量): {q3_avg_cost:.0f} 元/吨, {q3_annual_total:.0f} 吨/年')
285
286 return {
287     'q4_1_results': q4_1_results,
288     'q4_2_results': q4_2_storage_results,
289     'best_storage_cap': best_storage_cap,
290     'pareto_costs': pareto_costs,
291     'pareto_curtaills': pareto_curtaills,
292     'storage_caps': storage_caps.tolist(),
293     'comparison': comparison_data,
294     'scenarios': scenarios,
295 }
296
297 def plot_q4_results(q4_data, cfg):
298     scenarios = q4_data['scenarios']
299     q4_1 = q4_data['q4_1_results']
300
301     max_curtail = max(q4_1, key=lambda r: r['total_curtail'])
302     fig13 = plot_offgrid_power_balance(
303         HOURS, max_curtail['scenario']['P_wind'], max_curtail['scenario']['P_pv'],
304         cfg.P_conv, max_curtail['p_prod'], max_curtail['P_curtail'],
305         title=f'离网运行功率平衡 ({max_curtail["scenario"]["label"]}) '
306     )
307     save_fig(fig13, 'fig13_offgrid_power_balance.pdf', 'q4')
308
309     labels = [s['label'] for s in scenarios]
310     prods = [r['nh3_produced'] for r in q4_1]
311     curtaills = [r['total_curtail'] / max(1, np.sum(s['P_wind'] + s['P_pv']))
312                 for s, r in zip(scenarios, q4_1)]
313     fig14 = plot_production_vs_curtailment(
314         labels, prods, curtaills,
315         title='各场景产量与弃电分析 (离网无储能)'
316     )
317     save_fig(fig14, 'fig14_production_curtail.pdf', 'q4')
318
319     fig15 = plot_pareto_storage(
320         q4_data['storage_caps'], q4_data['pareto_costs'],
321         q4_data['pareto_curtaills'],
322         title='储能容量优化Pareto曲线'
323     )
324     save_fig(fig15, 'fig15_pareto_storage.pdf', 'q4')
325
326     if q4_data['best_storage_cap'] > 0:
327         rep_scenarios = [q4_1[0], q4_1[6], q4_1[12]]
328         soc_data = {}
329         for rs in rep_scenarios:
330             label = rs['scenario']['label']
331             for q4_2_r in q4_data['q4_2_results']:
332                 if q4_2_r['scenario']['id'] == rs['scenario']['id']:
333                     soc_data[label] = q4_2_r['soc'][:-1] / q4_data['best_storage_cap'] * 100
334                     break
335         if soc_data:
336             rep_q4_2 = [r for r in q4_data['q4_2_results'] if r['scenario']['id'] in [1, 7, 13]]
337             fig16 = plot_soc_discharge(
338                 soc_data,
339                 rep_q4_2[0]['P_ch'] if rep_q4_2 else np.zeros(24),
340                 rep_q4_2[0]['P_dis'] if rep_q4_2 else np.zeros(24),
341                 HOURS, title='储能SOC与充放电功率 (典型场景)'
342             )
343             save_fig(fig16, 'fig16_soc_discharge.pdf', 'q4')
344
345     with_storage = {
346         'curtail_rates': [r['total_curtail'] / max(1, np.sum(r['scenario']['P_wind'] + r['scenario']['P_pv']))
347                          for r in q4_data['q4_2_results']],
348         'productions': [r['nh3_produced'] for r in q4_data['q4_2_results']],
349         'util_rates': [r['nh3_produced'] / 72.0 for r in q4_data['q4_2_results']],
350         're_util_rates': [r['re_util'] for r in q4_data['q4_2_results']],
351     }
352     without_storage = {
353         'curtail_rates': [r['total_curtail'] / max(1, np.sum(r['scenario']['P_wind'] + r['scenario']['P_pv']))
354                          for r in q4_1],
355         'productions': [r['nh3_produced'] for r in q4_1],
356         'util_rates': [r['nh3_produced'] / 72.0 for r in q4_1],
357         're_util_rates': [r['re_util'] for r in q4_1],
358     }
359     fig17 = plot_offgrid_comparison(
360         scenarios, with_storage, without_storage,
361         title='离网运行有无储能对比'
362     )
363     save_fig(fig17, 'fig17_storage_comparison.pdf', 'q4')
364
365     fig18 = plot_net_vs_offgrid_comparison(
366         q4_data['comparison'],
367         title='离网与联网模式经济性对比'
368     )
369     save_fig(fig18, 'fig18_net_vs_offgrid.pdf', 'q4')
370
371 if __name__ == '__main__':
372     cfg = Config()
373     q4_data = run_q4()
374     plot_q4_results(q4_data, cfg)
375     print('\nQ4 figures saved.')

```

q5_policy.py

```
1 import numpy as np
2 import os, sys, json
3 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
4 from config import Config, FIG_DIR
5 from visualization import plot_policy_analysis, save_fig
6
7 def generate_policy_data():
8     cfg = Config()
9
10    policy_data = {
11        'negative_impacts': [
12            {
13                'title': '调峰压力显著增大',
14                'severity': 0.85,
15                'description': (
16                    '绿电园区高渗透率下，风电、光伏的间歇性和波动性将直接传递至公共电网。'
17                    '园区净负荷（总负荷-绿电出力）的随机波动幅度增大，'
18                    '导致电网调峰机组需频繁启停和大幅调节出力。'
19                    '以本园区为例，104MW新能源出力在0-104MW间波动，'
20                    '日最大峰谷差可达80MW以上。'
21                ),
22                'data': {
23                    'daily_peak_valley_MW': 85,
24                    'ramp_rate_MWh': 45,
25                }
26            },
27            {
28                'title': '备用容量需求增加',
29                'severity': 0.70,
30                'description': (
31                    '为应对绿电出力骤降（如云层遮挡导致光伏出力在10分钟内下降50%），'
32                    '电力系统需预留更多旋转备用和冷备用容量。'
33                    '研究表明，每增加10%可再生能源渗透率，系统备用容量需求增加3-5%。'
34                    '这将推高电力系统的运行成本和容量费用。'
35                ),
36                'data': {
37                    'reserve_increase_per_10pct_RE': 0.04,
38                    'annual_reserve_cost_increase': 15000000,
39                }
40            },
41            {
42                'title': '电能质量与系统稳定性挑战',
43                'severity': 0.60,
44                'description': (
45                    '大规模电力电子接口的新能源并网（逆变器、制氢电源等）'
46                    '将降低系统惯量，增加频率和电压越限风险。'
47                    '电解槽等非线性负荷会产生谐波污染，影响电能质量。'
48                    '当绿电渗透率超过30%时，系统的短路容量和惯量水平显著下降。'
49                ),
50                'data': {
51                    'max_RE_penetration_without_stability_issues': 0.30,
52                    'inertia_decline_per_10pct_RE': 0.08,
53                }
54            },
55        ],
56        'positive_impacts': [
57            {
58                'title': '促进新能源就近消纳',
59                'severity': 0.90,
60                'description': (
61                    '绿电直连园区通过"源网荷储"一体化模式，实现了新能源的就地消纳。'
62                    '本园区中，72t/d产能时绿电自用比例达86.5%，'
63                    '显著降低了新能源弃电率（6.7%），远低于全国平均弃风弃光率。'
64                    '这种模式为高比例新能源消纳提供了可复制的示范路径。'
65                ),
66                'data': {
67                    'self_consume_ratio': 0.865,
68                    'curtailment_rate': 0.067,
69                    'national_avg_curtailment': 0.15,
70                }
71            },
72            {
73                'title': '有效降低系统碳排放',
74                'severity': 0.95,
75                'description': (
76                    '绿电制氢氨替代传统化石能源制氢氨，具有显著的碳减排效益。'
77                    '每吨绿氨可减少CO2排放约2.5吨（对比煤气化制氨）。'
78                    '本园区满产年产量25920吨，年减排约64800吨CO2。'
79                    '同时，绿电直连模式减少了电网输电损耗，进一步提升能效。'
80                ),
81                'data': {
82                    'CO2_reduction_per_ton_nh3': 2.5,
83                    'annual_CO2_reduction_tons': 64800,
84                    'grid_loss_reduction': 0.03,
85                }
86            },
87            {
88                'title': '推动储能与氢能技术产业链发展',
89                'severity': 0.75,
90                'description': (
91                    '绿电直连园区对储能的需求（平抑波动、削峰填谷）'
92                    '将推动电池储能、氢储能等技术的规模化应用和成本下降。'
```

```

93         '同时, 绿氢-绿氨产业链的建立将带动电解槽、'
94         '合成氨催化剂、储氢材料等相关产业的发展。'
95         '形成"新能源-氢能-化工"联动的绿色产业集群。'
96     ),
97     'data': {
98         'battery_cost_decline_annual': 0.10,
99         'electrolyzer_cost_decline_annual': 0.08,
100     }
101 },
102 ],
103 'policy_recommendations': [
104     {
105         'title': '完善绿电市场化交易机制',
106         'priority': 0.92,
107         'description': (
108             '建立绿电直连园区的电力市场化交易机制, '
109             '包括绿电证书交易、碳排放权交易等。'
110             '允许园区参与辅助服务市场(调频、备用), '
111             '通过市场化手段获取灵活性收益。'
112             '建议设立绿电园区专用交易通道, 简化并网审批流程。'
113         ),
114     },
115     {
116         'title': '强制配套储能设施',
117         'priority': 0.82,
118         'description': (
119             '要求绿电直连园区按装机容量10-20%配置储能, '
120             '用于平滑出力波动和提供系统支撑。'
121             '储能容量配置可作为并网的前置条件。'
122             '对储能投资给予税收减免或补贴(如投资补贴30%)。'
123             '推动"共享储能"模式, 降低单一园区储能成本。'
124         ),
125     },
126     {
127         'title': '推广智能调度与预测技术',
128         'priority': 0.75,
129         'description': (
130             '应用风光功率预测、负荷预测、AI优化调度等技术, '
131             '实现园区与电网的协调运行。'
132             '建立园区级能量管理系统(EMS), '
133             '实现风光-制氢-储能-负荷的多时间尺度协调优化。'
134             '本研究表明, 连续调节(Q3)相比离散调节(Q2)'
135             '可降低吨氨成本10-30%, 提升绿电指标达标率。'
136         ),
137     },
138     {
139         'title': '实施差异化电价与补贴政策',
140         'priority': 0.88,
141         'description': (
142             '对绿电直连园区实行优惠的过网费政策, '
143             '降低"绿电-绿氨"产业链的综合用电成本。'
144             '对达到绿电指标要求的园区给予碳收益奖励。'
145             '设立绿电制氢氨专项基金, 支持技术研发和示范项目。'
146         ),
147     },
148     {
149         'title': '加强技术研发与标准制定',
150         'priority': 0.78,
151         'description': (
152             '支持大功率电解槽、高效合成氨催化剂、'
153             '低成本储氢材料等核心技术攻关。'
154             '制定绿电直连园区规划设计、运行管理、'
155             '并网接入等系列技术标准。'
156             '建立绿氢-绿氨认证体系, 打通国际市场准入。'
157         ),
158     },
159 ],
160 'quantitative_summary': {
161     'capacity_factor_range': '37.7%-100%',
162     'cost_range_yuan_per_ton': '3497-7435',
163     'CO2_reduction_annual_tons': 64800,
164     'RE_self_consumption_range': '82.4%-86.5%',
165     'storage_capex_per_kWh': 1000,
166     'recommended_storage_ratio': '10-20%',
167 }
168 }
169
170 return policy_data
171
172 def plot_q5_results(save_dir=None):
173     fig19 = plot_policy_analysis()
174     path = save_fig(fig19, 'fig19_policy_analysis.pdf', 'q5')
175     return path
176
177 def print_q5_summary():
178     data = generate_policy_data()
179     print('=' * 70)
180     print('问题五: 政策影响分析与建议')
181     print('=' * 70)
182
183     print('\n--- 不利影响 ---')
184     for imp in data['negative_impacts']:
185         print(f' [{imp["severity"]:.0%}] {imp["title"]}')
186         print(f' {imp["description"][:80]}...')
187
188     print('\n--- 有利影响 ---')

```

```

189     for imp in data['positive_impacts']:
190         print(f' [{imp["severity"]:.0%}] {imp["title"]}')
191         print(f'   {imp["description"][:80]}...')
192
193     print('\n--- 政策建议（按优先级） ---')
194     for rec in sorted(data['policy_recommendations'], key=lambda r: -r['priority']):
195         print(f' [{rec["priority"]:.0%}] {rec["title"]}')
196
197     print('\n--- 关键数据汇总 ---')
198     s = data['quantitative_summary']
199     print(f' 产能利用率范围: {s["capacity_factor_range"]}')
200     print(f' 吨氨成本范围: {s["cost_range_yuan_per_ton"]} 元/吨')
201     print(f' 年碳减排量: {s["CO2_reduction_annual_tons"]} 吨CO2')
202     print(f' 绿电自用比例: {s["RE_self_consumption_range"]}')
203     print('=' * 70)
204
205 if __name__ == '__main__':
206     print_q5_summary()
207     plot_q5_results()
208     print('\nQ5 figure saved.')

```

save_csv.py

```

1  import os, csv, numpy as np
2
3  TBL_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'tables')
4  os.makedirs(TBL_DIR, exist_ok=True)
5
6  def _w(filename, rows):
7      path = os.path.join(TBL_DIR, filename)
8      with open(path, 'w', newline='', encoding='utf-8-sig') as f:
9          writer = csv.writer(f)
10         for r in rows:
11             writer.writerow(r)
12     print(f' CSV: {filename}')
13     return path
14
15 def save_q1(q1, cfg):
16     rows = [
17         ['时段', '风电功率MW', '光伏功率MW', '常规负荷MW', '制氢氨功率MW',
18          '网购功率MW', '售电功率MW', '净负荷MW'],
19     ]
20     P_re = q1['P_wind'] + q1['P_pv']
21     net = P_re - q1['P_conv'] - q1['P_prod']
22     for t in range(24):
23         rows.append([
24             f'{t}:00', f'{q1["P_wind"][t]:.2f}', f'{q1["P_pv"][t]:.2f}',
25             f'{q1["P_conv"][t]:.2f}', f'{q1["P_prod"][t]:.2f}',
26             f'{q1["P_buy"][t]:.2f}', f'{q1["P_sell"][t]:.2f}', f'{net[t]:.2f}'
27         ])
28     _w('q1_power_balance.csv', rows)
29
30     ind = q1['indicators']
31     rows2 = [
32         ['指标', '数值', '要求', '达标'],
33         ['新能源发电量MWh', f'{q1["E_re"]:.2f}', '---', '---'],
34         ['总用电量MWh', f'{q1["E_total"]:.2f}', '---', '---'],
35         ['上网电量MWh', f'{q1["E_sell"]:.2f}', '---', '---'],
36         ['网购电量MWh', f'{q1["E_buy"]:.2f}', '---', '---'],
37         ['自发自用比例', f'{ind["self_consume_ratio"]:.2%}', '>=60%',
38          '达标' if ind['self_consume_pass'] else '不达标'],
39         ['总用电绿电比例', f'{ind["green_ratio"]:.2%}', '>=30%',
40          '达标' if ind['green_ratio_pass'] else '不达标'],
41         ['上网电量比例', f'{ind["curtail_ratio"]:.2%}', '<=20%',
42          '达标' if ind['curtail_pass'] else '不达标'],
43         ['吨氨成本元吨', f'{q1["cost_per_ton"]:.2f}', '---', '---'],
44     ]
45     _w('q1_indicators.csv', rows2)
46
47 def save_q2(q2, cfg):
48     levels = q2['production_levels']
49     typ = q2['typical_results']
50     rows = [['日产量t_d', '运行小时h', '吨氨成本元_t', '自用比例',
51             '绿电比例', '上网比例', '全达标']]
52     for l in levels:
53         r = typ[l]
54         ind = r['indicators']
55         rows.append([
56             str(l), str(r['hours_run']), f'{r["cost_per_ton"]:.2f}',
57             f'{ind["self_consume_ratio"]:.2%}', f'{ind["green_ratio"]:.2%}',
58             f'{ind["curtail_ratio"]:.2%}', '是' if ind['all_pass'] else '否'
59         ])
60     _w('q2_typical.csv', rows)
61
62     scens = q2['scenarios']
63     all_r = q2['all_results']
64     rows2 = [['场景'] + [f'{l}t_d' for l in levels]]
65     for i, s in enumerate(scens):
66         row = [s['label']]
67         for l in levels:

```

```

68         if i < len(all_r.get(1, [])):
69             row.append(f'{all_r[1][i]["cost_per_ton"]:.0f}')
70         else:
71             row.append('')
72         rows2.append(row)
73     _w('q2_scenario_costs.csv', rows2)
74
75     rows3 = [['日产量t_d', '全满足', '部分满足', '全不满足']]
76     for l in levels:
77         c = q2['compliance_counts'][l]
78         rows3.append([str(l), str(c['all_pass']), str(c['partial_pass']), str(c['no_pass'])])
79     _w('q2_compliance.csv', rows3)
80
81     def save_q3(q3, cfg):
82         levels = q3['production_levels']
83         typ = q3['q3_typical']
84         rows = [['日产量t_d', '实产t', '吨氨成本元_t', '自用比例', '绿电比例', '上网比例']]
85         for l in levels:
86             r = typ[l]
87             ind = r['indicators']
88             rows.append([
89                 str(l), f'{r["nh3_production"]:.1f}', f'{r["cost_per_ton"]:.2f}',
90                 f'{ind["self_consume_ratio"]:.2%}', f'{ind["green_ratio"]:.2%}',
91                 f'{ind["curtail_ratio"]:.2%}'
92             ])
93     _w('q3_typical.csv', rows)
94
95     rows2 = [['日产量t_d', '全满足', '部分满足', '全不满足']]
96     for l in levels:
97         c = q3['compliance_counts'][l]
98         rows2.append([str(l), str(c['all_pass']), str(c['partial_pass']), str(c['no_pass'])])
99     _w('q3_compliance.csv', rows2)
100
101     def save_q4(q4, cfg):
102         from utils import generate_24_scenarios
103         scens = q4['scenarios']
104         q4_1 = q4['q4_1_results']
105         rows = [['场景', '离网产量t', '弃电MWh', '可再生利用率', '吨氨成本元_t']]
106         for i, r in enumerate(q4_1):
107             rows.append([
108                 scens[i]['label'], f'{r["nh3_produced"]:.1f}',
109                 f'{r["total_curtail"]:.1f}', f'{r["re_util"]:.1%}',
110                 f'{r["cost_per_ton"]:.0f}'
111             ])
112     _w('q4_1_offgrid.csv', rows)
113
114     comp = q4['comparison']
115     rows2 = [
116         ['模式', '吨氨成本元_t', '日产量t', '年产量t', '产能利用率'],
117         ['离网无储能', f'{comp["offgrid"]["吨氨成本"]:.0f}',
118          f'{comp["offgrid"]["年产量"]/360:.1f}',
119          f'{comp["offgrid"]["年产量"]:.0f}',
120          f'{comp["offgrid"]["年产量"]/72/360:.1%}'],
121         ['联网同等产量', f'{comp["ongrid"]["吨氨成本"]:.0f}',
122          f'{comp["ongrid"]["年产量"]/360:.1f}',
123          f'{comp["ongrid"]["年产量"]:.0f}',
124          f'{comp["ongrid"]["年产量"]/72/360:.1%}'],
125     ]
126     _w('q4_3_comparison.csv', rows2)
127
128     def save_all(q1=None, q2=None, q3=None, q4=None, cfg=None):
129         if q1 is not None:
130             save_q1(q1, cfg)
131         if q2 is not None:
132             save_q2(q2, cfg)
133         if q3 is not None:
134             save_q3(q3, cfg)
135         if q4 is not None:
136             save_q4(q4, cfg)

```

run_all.py

```

1  import sys, os
2  import time
3
4  base = os.path.dirname(os.path.abspath(__file__))
5  if base not in sys.path:
6      sys.path.insert(0, base)
7
8  from config import Config
9  import numpy as np
10
11  def main():
12      print('=' * 70)
13      print('绿电直连型电氢氨园区优化运行 - 全部分析')
14      print('=' * 70)
15      total_start = time.time()
16
17      cfg = Config()
18
19      print('\n' + '=' * 70)

```

```

20 print('问题一: 典型风光场景分析')
21 print('=' * 70)
22 t0 = time.time()
23 from q1_simulation import run_q1, plot_q1_results, print_q1_results
24 q1_results = run_q1()
25 print_q1_results(q1_results, cfg)
26 plot_q1_results(q1_results, cfg)
27 print(f' [完成 用时{time.time()-t0:.1f}s]')
28
29 print('\n' + '=' * 70)
30 print('问题二: 离散制氮调节优化')
31 print('=' * 70)
32 t0 = time.time()
33 from q2_scheduling import run_q2, plot_q2_results
34 q2_data = run_q2()
35 plot_q2_results(q2_data, cfg)
36 print(f' [完成 用时{time.time()-t0:.1f}s]')
37
38 print('\n' + '=' * 70)
39 print('问题三: 连续制氮调节优化')
40 print('=' * 70)
41 t0 = time.time()
42 from q3_continuous import run_q3, plot_q3_results
43 q3_data = run_q3()
44 plot_q3_results(q3_data, cfg, q2_data)
45 print(f' [完成 用时{time.time()-t0:.1f}s]')
46
47 print('\n' + '=' * 70)
48 print('问题四: 离网运行及储能配置')
49 print('=' * 70)
50 t0 = time.time()
51 from q4_offgrid import run_q4, plot_q4_results
52 q4_data = run_q4()
53 plot_q4_results(q4_data, cfg)
54 print(f' [完成 用时{time.time()-t0:.1f}s]')
55
56 print('\n' + '=' * 70)
57 print('问题五: 政策分析')
58 print('=' * 70)
59 t0 = time.time()
60 from q5_policy import print_q5_summary, plot_q5_results
61 print_q5_summary()
62 plot_q5_results()
63 print(f' [完成 用时{time.time()-t0:.1f}s]')
64
65 print('\n' + '=' * 70)
66 print('导出CSV结果文件')
67 print('=' * 70)
68 from save_csv import save_all
69 save_all(q1_results, q2_data, q3_data, q4_data, cfg)
70
71 total_time = time.time() - total_start
72 print(f'\n 总计用时: {total_time:.1f}s')
73
74 fig_dir = os.path.join(base, '..', 'figures')
75 all_figs = []
76 for root, dirs, files in os.walk(fig_dir):
77     for f in files:
78         if f.endswith('.pdf'):
79             all_figs.append(os.path.relpath(os.path.join(root, f), fig_dir))
80 tbl_dir = os.path.join(base, '..', 'tables')
81 all_csvs = [f for f in os.listdir(tbl_dir) if f.endswith('.csv')] if os.path.isdir(tbl_dir) else []
82 print(f'\n共生成 {len(all_figs)} 个图表文件 + {len(all_csvs)} 个CSV结果文件')
83 for f in sorted(all_figs):
84     print(f' 图 {f}')
85 for f in sorted(all_csvs):
86     print(f' CSV {f}')
87
88 if __name__ == '__main__':
89     main()

```